

Communication and Runtime Engineering for Two-Party ResNet Inference in HPMPC

ABY2 with HE and silent-OT preprocessing

Technical report, HPMPC dbg branch

hpmc 97772ef, ConvTriple fa7b952, flexNN b1ce619; ImageNet all_opt (Section 9): hpmc 92a2c0e, ConvTriple 8d8f244 (GPU,

October 2, 2026

Abstract

We describe the optimizations and fixes that make HPMPC’s two-party ABY2 inference of ResNet50 fast, reproducible and correct across all of its protocol variants. The work targets both communication and runtime, in the preprocessing phase (homomorphic encryption for linear layers, silent OT for Boolean material) and in the online phase (round latency and per-round work). For one ImageNet image, preprocessing drops from 17.1 s to 4.3 s on two AMD EPYC 7543 hosts (3.3 s on EPYC 9354) and its triple traffic from 1112 to 587 MiB. The online phase drops from 2.4–4.3 s to 0.70 s. BatchNorm triples shrink from 5.7 GB to 0.19 GB of traffic per 10 CIFAR-10 images, and Beaver 3- and 4-tuples need half the OTs and about 45% less traffic. Evaluating several conv-triple chunks at once halves the CIFAR-10 conv phase, and in multi-batch mode the eight lanes of a register share one convolution, which cuts the conv phase per process from 8.6 to 3.9 s and the preprocessing traffic by half. On CIFAR-10 we evaluate 30 protocol variants (three MSB adders, five ReLU families, fused and unfused BatchNorm) in single-batch and 192-image multi-batch mode on two machine pairs. On the EPYC 9354 pair, preprocessing takes 1.3–2.7 s per 10 images and the online phase 0.35–0.50 s; 192 images take 7.8–19.5 s of preprocessing. Public weights, which need no linear-layer triples, and fully shared weight masks (A_KNOWN=0) work in every variant. With the AdamW-trained ResNet50, every variant classifies 62–72 of 100 images (plaintext 76) and 118–131 of 192 (61–68%; plaintext 71.9%). Along the way we fixed a thread race, two connection hangs, four protocol bugs that broke whole variant families, and the sources of run-to-run nondeterminism: all 60 threaded builds reproduced their serial reference bit for bit, and the final single-batch builds still give the same outputs. For the artifact’s 36 single-batch all_opt configurations on ImageNet, preprocessing gets 2.9–7.1× faster with secret weights and the online phase 2.0–3.0× (EPYC 9354). Their A2bits configurations were not correct as given; the A2B bake that fixes them commits every ReLU input’s mask before the preprocessing pass, for free behind convolutions and residual sums and through a mask-only forward with public weights, and an optional output repacking cuts the conv triples’ traffic by 40%. On a GPU, the packed pipeline’s evaluator produces the conv triples 2.1–2.3× faster than the CPU and 5.6–7.0× faster than the previous GPU path. With ferret’s extensions and the random OTs’ hashing on the GPU as well, bit for bit emp’s, preprocessing on one machine shared by both parties is 1.7–2.2× faster than on its CPU.

1 Introduction

Two-party inference with a preprocessing phase splits the cost of a secure neural network into two parts. An input-independent *preprocessing phase* produces correlated randomness: multiplication triples for the linear layers, generated with homomorphic encryption (HE), and Boolean triples and tuples for the nonlinear layers, generated with oblivious transfer (OT). The *online phase* then evaluates the network on masked values and needs only a few messages per layer. HPMPC implements this design with an ABY2-style protocol [1] (PROTOCOL=4) and generates its preprocessing material with the CHEETAH [2] HE and silent-OT [3] machinery (ConvTriple).

This report collects the engineering work that took this pipeline from a working prototype to a fast and reproducible system. Its focus is communication and runtime; Fig. 2 summarizes the largest effects.

- **HE preprocessing** (Section 3): a packed and pipelined convolution evaluator, an exact wire format that sends only the bits decryption needs, and BatchNorm triples packed into full ciphertexts and generated for all layers in one product, several chunks evaluated at once, and in multi-batch mode one convolution for all lanes. ImageNet preprocessing: 17.1 s $\rightarrow$ 4.3 s (4.0×).
- **Boolean preprocessing** (Section 4): OT packs sized by total demand, ferret parameters and a prefetching LPN step, TCP_NODELAY on every channel, bit-packed random OTs, and Beaver tuples whose products share one OT per common factor (half the OTs, $\approx$ 45% less traffic).

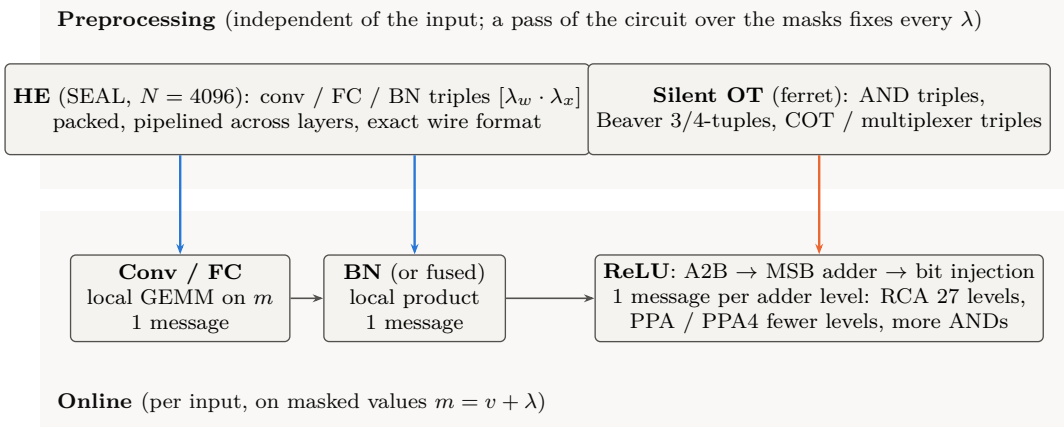


Figure 1: Where the cost of 2PC inference arises. A preprocessing pass fixes every mask; HE produces the linear-layer triples and silent OT the Boolean material. Online, a linear layer costs one message and a ReLU one message per level of its MSB adder.

- **Online phase** (Section 5): Nagle-free sockets, element-parallel circuit levels for batch size one, pooled GEMM threads, an ahead-of-time PRNG and fewer rounds per ReLU. ImageNet online: $2.4\text{--}4.3\text{ s} \rightarrow 0.70\text{ s}$.
- **Correctness and reproducibility** (Section 7): deterministic OT and HE randomness, and fixes for a worker race, lost wake-ups, stale listeners, multi-batch A2B, the threaded GEMM, and the BatchNorm of $A_KNOWN=0$ and of public weights.
- **Evaluation** (Section 8): 30 variants $\times$ 2 batch modes $\times$ 2 machine pairs, with final accuracy on a trained model.
- **The all_opt configurations on ImageNet** (Section 9): 36 builds over three weight settings, two ReLU protocols and three adders. The A2bits configurations as given compute wrong signs; the A2B bake fixes them by committing every ReLU input’s mask before the preprocessing pass, at no traffic behind convolutions and residual sums. The conv triples start alongside the OT phase. Preprocessing $2.9\text{--}7.1\times$ faster with secret weights, online $2.0\text{--}3.0\times$.
- **GPU** (Section 3.3): a GPU evaluator inside the packed conv pipeline, and ferret’s LPN step, GGM trees and the random OTs’ hashing on the GPU, all bit-identical to the CPU’s. Conv triples $2.1\text{--}2.3\times$ faster; end to end, on one shared machine, preprocessing $1.7\text{--}2.2\times$ faster than on the CPU.

2 Setting

Protocol. ABY2 shares a value v as a public masked value $m = v + \lambda$ together with additive shares of the mask $\lambda = \lambda_0 + \lambda_1$. Every mask is fixed in preprocessing, so a linear layer is local except for one message that re-masks its output, and its preprocessing is a triple over the masks, e.g. $[\lambda_w \cdot \lambda_x]$ for a convolution (Fig. 1). With $A_KNOWN=1$ the model owner P0 holds the weight masks and the data owner P1 the activation masks (the AB2 triple); with $A_KNOWN=0$ both are shared (AB). We name the three weight settings by who knows the weights: *UC1*, known to none (secret-shared, $A_KNOWN=0$); *UC2*, known to one party, the model owner ($A_KNOWN=1$); *UC3*, known to all (public weights, $PUBLIC_WEIGHTS=1$, no linear-layer triples). A ReLU converts its input to Boolean shares (A2B), extracts the sign with an MSB adder and injects the bit back into the arithmetic domain (bit injection). The adder sets the number of online rounds per ReLU: the ripple-carry adder (RCA) needs 27 AND levels, the parallel-prefix adders PPA and PPA4 (4-input prefix gates, which use Beaver 3- and 4-tuples) need fewer levels but more AND gates.

Variants. We measure three adders (RCA, PPA, PPA4) and five ReLU families: *plain*; *reshare* ($RESHARE_OPT$); *reshare+sim*, which folds the reshare material into the convolution mask ($RESHARE_OPT_SIM$); *a2b*, which moves the A2B to preprocessing and bakes its mask into the convolution output ($A2B_ONLINE_OPT$, $A2B_CONV_BAKE$); and *a2b+AKTE*, an adder for inputs whose mask the evaluators know ($A_KNOWN_TO_EVALUATORS_OPT$). Each variant runs with BatchNorm fused into the preceding convolution or unfused. All variants share $BITLENGTH=32$, $FRACTIONAL=5$, probabilistic (SecureML-style) truncation, $FUSE_RELU_AVG=1$ and $CUT_FRACTIONAL_BITS_OPT=1$.

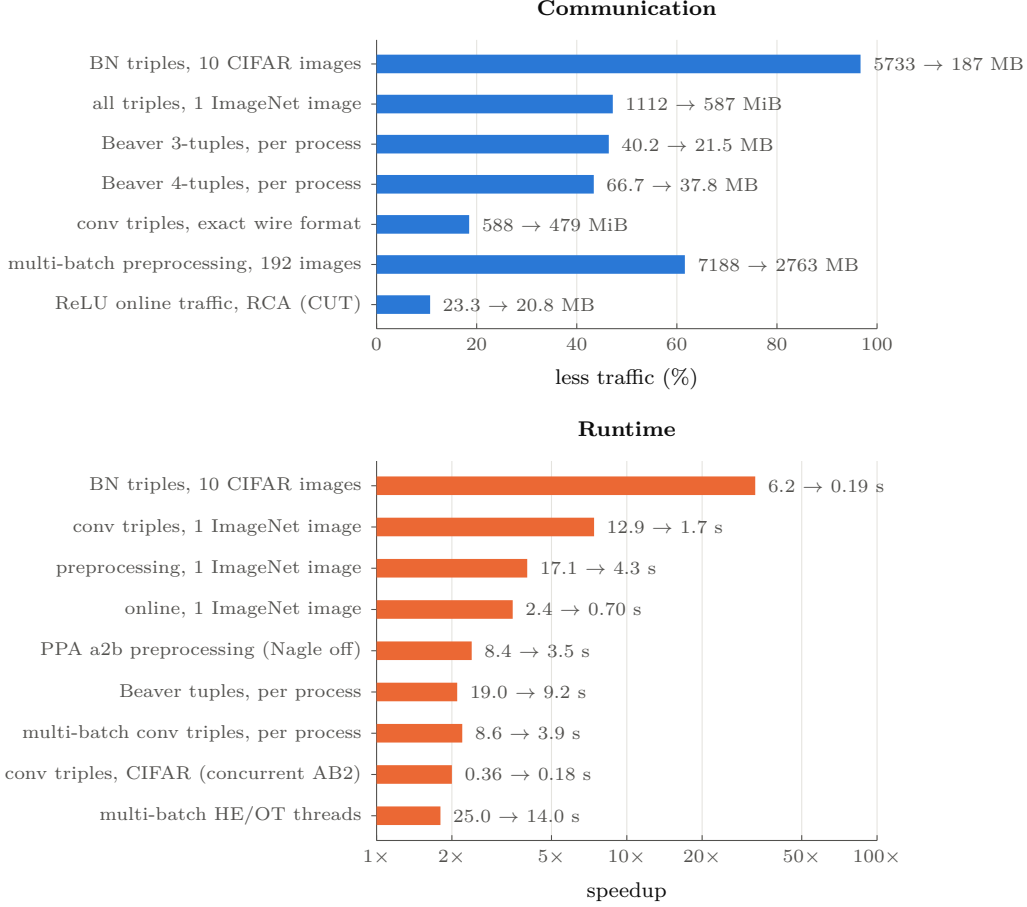


Figure 2: Headline effects of the changes in this report, each labeled with its before and after values. Communication: share of traffic removed. Runtime: speedup (log scale). Sections 3–6 explain each.

Workloads. ResNet50 on CIFAR-10 in two modes. *Single batch*: one process, 32-bit lanes (`DATTYPE=32`), 10 images, 32 HE/OT threads and 31 extra online threads. *Multi-batch*: 24 processes with 256-bit registers of eight 32-bit lanes, 192 images, 3 HE/OT threads and one extra online thread per process. The HE preprocessing study and the online steps use ResNet50 on one ImageNet image, `FUNCTION_IDENTIFIER=87`: CHEETAH’s ResNet50 layout for 1000 classes, whose 224×224 input arrives padded to 230×230 . It runs with a dummy model and image; since no code path depends on the values (Section 3.2), the times are those of a trained model. Table 1 lists every flag that differs from the defaults in `config.h`.

Measurement. Preprocessing and online time are P0’s wall-clock times (the maximum over processes in multi-batch). Single-batch numbers are medians of two runs; multi-batch runs once. All randomness is seeded, so every run of a build produces a bit-identical output; we check each run’s output hash against a serial build without extra threads (Section 7). The runtime does not depend on the weights: HE triples are generated with `TRIPLE_ZERO=0N` and no code path skips zero values. Accuracy uses two trained ResNet50 models: `Cifar_adam_001` for the runtime matrix and an AdamW-trained model (74.35% on the test set) for the final accuracy.

3 Linear layers: HE preprocessing

3.1 Convolution triples: construction and security

This subsection defines the convolution triple, the packed encoding and the protocol that generates it, and argues correctness, security and cost. The engineering steps that follow (Fig. 4) change how fast the protocol runs, not what it sends.

Notation. All arithmetic shares live in $\mathbb{Z}_{2^{32}}$. BFV works over $R_t = \mathbb{Z}_t[X]/(X^N + 1)$ with $N = 4096$ and plaintext modulus $t = 2^{32}$, so a plaintext coefficient is exactly one ring element. The ciphertext modulus is

ImageNet runs	flags beyond the <code>config.h</code> defaults
Model, input Preprocessing	FUNCTION_IDENTIFIER=87 (default 15), NUM_INPUTS=1, DATATYPE=32 (one image, one process) PRE=1, CHEETAH_CONV_PACKED=1 (from s1 on; s0 is CHEETAH's HomConv2DSS), CHEETAH_CONV_PIPELINE=1 (the default with packing, from s8 on), CHEETAH_THREADS=32
Network layers	FUSE_CONV_BN=1, FUSE_RELU_AVG=1, RCA_MSB=1
Roles	MODELOWNER=P_0, DATAOWNER=P_1 (secret weights: A_KNOWN=1, PUBLIC_WEIGHTS=0, TRUNC_DELAYED=0 and MODELWEIGHTS_KNOWN_DURING_PREPROCESSING=0 were set explicitly, equal to the defaults)
ConvTriple (CMake)	TRIPLE_ZERO=ON; TRIPLE_FERRET=b12 from s10 on (b13 before)
Online steps (Fig. 10)	in addition CUT_FRACTIONAL_BITS_OPT=1, SEND_BUFFER=100000, RECV_BUFFER=100000, ADDITIONAL_GEMM_THREADS=31, ADDITIONAL_RELU_THREADS=31, RNG_AHEAD=1

Table 1: Build configuration of the ImageNet measurements. The other flags keep their defaults, e.g. `PROTOCOL=4`, `BITLENGTH=32`, `FRACTIONAL=5`.

	flare / polynize	algofi / goracle
CPU	AMD EPYC 9354 (Zen 4), AVX-512, VAES, IFMA	AMD EPYC 7543 (Zen 3), AVX2, AES-NI
Cores, memory	32 cores / 64 threads, 752 GB	32 cores / 64 threads, 512 GB
Link	25 Gbit/s direct (Intel E810, ice 2.6.7), MTU 9700, 24.7 Gbit/s iperf3, RTT 0.28–0.31 ms	
Roles	P0 (model owner): flare / algofi; P1 (data owner): polynize / goracle	

Table 2: Machines. Each host builds its own binaries (`-march=native`).

$q = q_0 q_1$ with primes of 60 and 49 bits (109 bits, 128-bit security by the HE standard's table), $\Delta = \lfloor q/t \rfloor$, s is a party's ternary secret key and χ the error distribution. Enc_s denotes secret-key and Enc_{pk} public-key encryption. A convolution $y = W \star x$ is computed after the polyphase reduction of `ConvLayout`: a stride- σ , padded convolution is rewritten as a stride-1, unpadded convolution over σ^2 phase channels, which yields exactly the strided outputs. We therefore only treat stride-1 valid convolutions of B images of $C \times H \times W$ with M filters of $C \times k_h \times k_w$.

The triple, and why AB2 halves the traffic. With masked values $m_v = v + \lambda_v$, the ABY2 online step of a convolution is

$$m_y = m_W \star m_x - m_W \star \lambda_x - \lambda_W \star m_x + \Gamma + \lambda_y, \quad \Gamma = \lambda_W \star \lambda_x,$$

computed share-wise (P0 adds $m_W \star m_x$). The preprocessing must deliver additive shares of Γ , the *convolution triple*. Write $\lambda_v = \lambda_v^0 + \lambda_v^1$ for the parties' shares.

- **AB** (`A_KNOWN=0`). Both masks are shared, and $\Gamma = \lambda_W^0 \star \lambda_x^0 + \lambda_W^1 \star \lambda_x^1 + \lambda_W^0 \star \lambda_x^1 + \lambda_W^1 \star \lambda_x^0$. The first two terms are local; each cross term needs one secure product in which one party holds a filter and the other an input: two HE products per layer, one in each direction.
- **AB2** (`A_KNOWN=1`). The model owner P0 holds the whole weight mask ($\lambda_W^1 = 0$), and $\Gamma = \lambda_W \star (\lambda_x^0 + \lambda_x^1)$ is a *single* product: P1 encrypts λ_x^1 , P0 adds its own λ_x^0 inside the ciphertext and multiplies by its plaintext filters.

AB2 therefore needs half the ciphertexts, half the bytes and half the HE work of AB: 479 against 958 MiB for the ImageNet convolutions and 99 against 197 MiB for 10 CIFAR-10 images (Table 6), with the same protocol for each product. It is secure because it is ABY2's own input semantics: the party that inputs a value chooses that value's whole mask. The weights are P0's input. With `SHARE_PREP` P0 sets $\lambda_W = -W$, so the public masked weight is $m_W = 0$ and P1's share is 0: P1 learns nothing about W , and P0 learns only a mask of its own input. Nothing about x reaches P0, since λ_x^1 stays encrypted under P1's key. AB is needed only when no single party owns the weights (e.g. a secret-shared model); with the weights known in preprocessing (`MODELWEIGHTS_KNOWN_DURING_PREPROCESSING`) the same single product is used, with P1's share of Γ prescribed by its PRNG stream instead of drawn fresh, so no extra message is needed.

Packed encoding. The encoding follows the block layout of troy's `Conv2dHelper`. A tiling is (b, c_i, c_o, h, w) with $b c_i c_o h w \leq N$, $h \geq k_h$, $w \geq k_w$; a tile covers b images and an $h \times w$ window, and yields $y_h \times y_w = (h - k_h + 1) \times (w - k_w + 1)$ outputs per image and filter. For a tile τ (first image β_0 , first row r_0 , first column

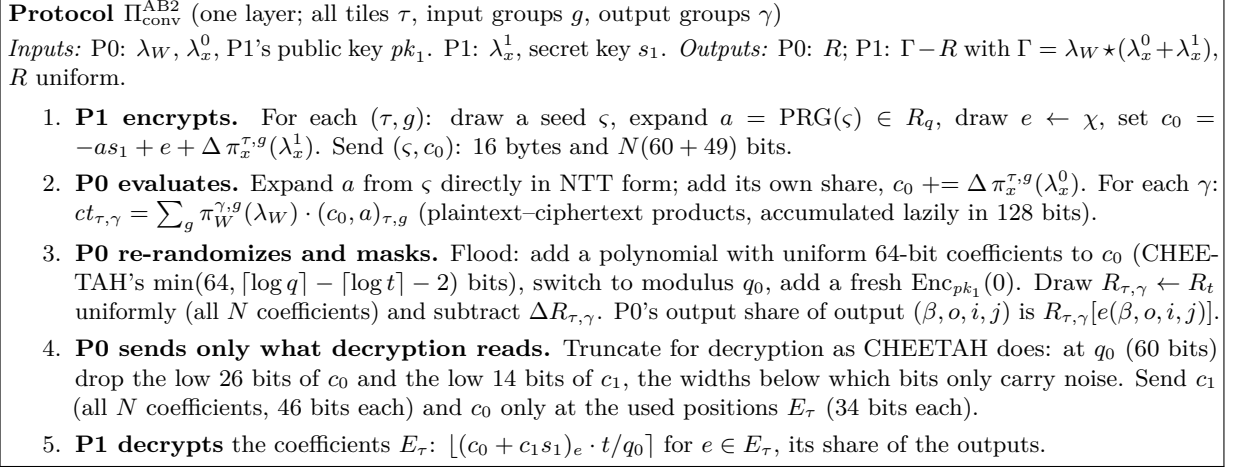


Figure 3: The AB2 convolution-triple protocol (PackedConv2D). AB runs it twice, once in each direction, with P0's and P1's roles swapped for the second cross term.

s_0) and an input group g of c_i channels, and an output group γ of c_o filters,

$$\pi_x^{\tau, g}(X) = \sum_{\beta < b} \sum_{c < c_i} \sum_{i < h} \sum_{j < w} x[\beta_0 + \beta, gc_i + c, r_0 + i, s_0 + j] X^{(\beta c_i c_o + c)hw + iw + j},$$

$$\pi_W^{\gamma, g}(X) = \sum_{o < c_o} \sum_{c < c_i} \sum_{a < k_h} \sum_{d < k_w} W[\gamma c_o + o, gc_i + c, k_h - 1 - a, k_w - 1 - d] X^{(oc_i + c_i - 1 - c)hw + aw + d}.$$

Lemma 1. For $\beta < b, o < c_o, i < y_h, j < y_w$ let $e(\beta, o, i, j) = (\beta c_i c_o + oc_i + c_i - 1)hw + (k_h - 1 + i)w + (k_w - 1 + j)$. Then $[\pi_x^{\tau, g} \cdot \pi_W^{\gamma, g}]_{e(\beta, o, i, j)} = \sum_{c < c_i} \sum_{a < k_h} \sum_{d < k_w} x[\beta_0 + \beta, gc_i + c, r_0 + i + a, s_0 + j + d] \cdot W[\gamma c_o + o, gc_i + c, a, d]$ in R_t , i.e. the input group's partial sum of output $(\beta_0 + \beta, \gamma c_o + o, r_0 + i, s_0 + j)$, and $\sum_g \pi_x^{\tau, g} \pi_W^{\gamma, g}$ holds the full output there.

Proof sketch. A product term of input (β, c, i', j') and flipped tap (o, c', a, d) has exponent $(\beta c_i c_o + oc_i + c_i - 1 + c - c')hw + (i' + a)w + (j' + d)$. (i) The column offset $j' + d \leq w + k_w - 2$ never reaches $w + k_w - 1 + j$, so no term carries from the previous row onto an output position, and the row offset is at most $(h + k_h - 2)w + w + k_w - 2$, so a carry from the previous block lands below $(k_h - 1)w + k_w - 1$, the smallest output offset. (ii) Within the target block $\beta c_i c_o + oc_i + c_i - 1$ only $c = c'$ contributes; $i' + a = k_h - 1 + i$ and $j' + d = k_w - 1 + j$ select exactly the taps of the valid convolution after the flip. (iii) Exponents reach N only for $\beta = b - 1, o = c_o - 1, c > c'$, i.e. blocks $bc_i c_o + (0 \dots c_i - 2)$; negacyclic reduction maps them to blocks $0 \dots c_i - 2$ (with a sign), and output blocks are $\equiv c_i - 1 \pmod{c_i}$, so no wrapped term reaches an output coefficient. $\square$

The remaining coefficients hold partial sums of other channel pairings; they are masked like the outputs and never decrypted. The conv-triple test (`cheetah_conv_triple_test`) checks every output exactly against a plaintext convolution, for the 23 CIFAR-10 layer shapes, the ImageNet layers and randomized shapes.

Tiling. A layer needs $T = \lceil B/b \rceil \lceil (H - k_h + 1)/y_h \rceil \lceil (W - k_w + 1)/y_w \rceil$ tiles, $G_{\text{in}} = \lceil C/c_i \rceil$ input ciphertexts and $G_{\text{out}} = \lceil M/c_o \rceil$ output ciphertexts per tile. The tiling minimizes the bytes on the wire (below),

$$\min_{b, h, w, c_o} T \cdot (G_{\text{in}} \cdot \text{in} + G_{\text{out}} \cdot \text{out}(b c_o y_h y_w)), \quad c_i = \min(\lfloor N/(b c_o h w) \rfloor, C),$$

where in and $\text{out}(u)$ are the sizes of an input ciphertext and of an output ciphertext with u used coefficients. CHEETAH's HomConv2DSS fixes $c_o = 1$: one output ciphertext per filter and tile. On ImageNet's 7×7 stage, such a ciphertext carries 49 useful coefficients out of 4096; the packed layout puts up to $\lfloor N/(b h w) \rfloor$ filters and several images into one output ciphertext.

Security. Theorem 1. $\Pi_{\text{conv}}^{\text{AB2}}$ securely computes the functionality that outputs a uniformly random R to P0 and $\Gamma - R$ to P1 against a semi-honest adversary corrupting one party, assuming decision-RLWE for (N, q, χ) , the seed expansion modeled as a random oracle, and the statistical circuit privacy of CHEETAH's flooding with its parameters.

Proof sketch. **Corrupted P0.** Its view is the pairs (ς, c_0) . With $a = \text{PRG}(\varsigma)$ modeled as a random oracle, a is uniform and public, which is exactly the RLWE setting (SEAL's seeded ciphertexts rely on the same argument). A hybrid over the ciphertexts replaces each $c_0 = -as_1 + e + \Delta m$ by a uniform element; the simulator sends uniform (ς, c_0) . **Corrupted P1.** Its output $\Gamma - R$ is uniform because R is. Before truncation, the ciphertext

ImageNet ResNet50, one image	triples, P0 sent + received (MiB)	factor
CHEETAH HomConv2DSS ($c_o = 1$), SEAL serialization (s0)	1,112	1.00
packed layout (s1)	698	0.63
+ exact wire format, byte-optimal tiling (s9, s10)	587	0.53
conv triples only, final: AB2 (A_KNOWN=1) / AB (A_KNOWN=0)	479 / 958	1 / 2

Table 3: Where the conv-triple traffic goes (UC2, 32-bit ring). The first three rows include the Boolean and FC material (108 MiB, unchanged); the last row compares the two triple types with the same protocol.

P1 receives is, by the flooding and the fresh encryption of zero, statistically close to a fresh $\text{Enc}_{pk_1}(\Pi)$ of the plaintext $\Pi = P - R$, where P is the product polynomial. Since R covers all N coefficients, Π is uniform in R_t : its used coefficients equal P1’s output and the others are independent and uniform. The simulator therefore samples Π uniform subject to P1’s output on E_τ , encrypts it with flooding, and applies the same deterministic truncation and selection of c_0 coefficients. The changes of this work leave this argument intact: the layout only changes where values sit (the tiling depends on public shapes only), and the exact wire format and the partial c_0 are deterministic functions of the message CHEETAH would send, so they cannot reveal more. The bits that are not sent are either zero after truncation (checked at run time) or coefficients that decryption never reads. Finally, the evaluator’s work does not depend on the weights’ values (Section 3.2), so its running time reveals nothing about zero weights, which matters when the weights are known in preprocessing. $\square$

Communication. An input ciphertext costs $in = 16 + N(60 + 49)/8 = 55,824$ bytes, against 62.4 KB as a zstd-compressed SEAL seeded ciphertext; an output ciphertext costs $out(u) = \lceil 34u/8 \rceil + 46N/8 = 23,552 + 4.25u$ bytes. For each layer, the tiling minimizes $T(G_{in} in + G_{out} out)$; since an input costs about twice an output, it favors fewer inputs. Two effects add up on ImageNet (Table 3). Packing several filters and images into an output ciphertext removes most of the output ciphertexts of the late stages, and the exact wire format then cuts the remaining bytes. Together with AB2, the conv triples of a ResNet50 inference need 479 MiB, about a quarter of the roughly 2,000 MiB of AB with CHEETAH’s layout (s0 with A_KNOWN=0: 2,115 MiB of triples in total).

What is left. A model of the layout search and the wire format (`docs/paper/he_model.py`) reproduces the measured 244.1 MiB of input and 234.9 MiB of output ciphertexts of the ImageNet convolutions. Dense packing in the same format would need 234.7 MiB, so about half of the traffic is unused coefficients. At 56×56 a channel fills 3,136 of 4,096 coefficients (77%). At 14×14 and 7×7 the layout has to trade inputs against outputs: a polynomial holds c_i input channels and c_o output-channel blocks, so inputs use about $1/c_o$ and outputs about $1/c_i$ of their coefficients (10–48% and 4–24% in the chosen layouts). Stages 3 and 4 send 237 MiB where dense packing would need 67 MiB. Three cheap levers do not help. A per-layer ring dimension $N = 8192$ (same 109-bit modulus) is never cheaper, except on the first layer at equal cost. Fewer bits of the input ciphertexts would raise the product noise, which P1 knows and the 64-bit flooding must hide: with 12 bits of headroom below $\Delta = 2^{77}$, at most about 16 of 109 bits could go, paid for with more output bits. The output format already keeps only the bits decryption needs.

The remaining lever is *output repacking*: the evaluator packs the used coefficients of several sparse output ciphertexts into dense ones with ring automorphisms (a trace per ciphertext, or PackLWEs-style merging). That needs key switching, and key switching needs a special prime on top of the 109-bit data modulus ($60 + 49$ bits: 2^{32} plaintexts and 64-bit flooding). At $N = 4096$ the 128-bit security bound is already exhausted by those 109 bits, so repacking forces $N = 8192$ (with a 60-bit special prime, 169 of at most 218 bits). With the layout free to use dense inputs ($c_o = 1$) and outputs packed after the evaluation, the model gives 134 MiB of inputs and 108 MiB of outputs, 243 instead of 479 MiB per HE product, plus 4.7 MiB of Galois keys that P1 sends once (13 keys, SEAL serialization). Keeping today’s layouts and repacking only the outputs would give just 420 MiB. Over the whole preprocessing (`repack_estimate.py`, measured traffic of the other components) this removes 22–38% of the traffic in UC1, whose AB triples need both products (1,230–2,150 to 766–1,686 MiB), and 14–29% in UC2 (790–1,710 to 558–1,478 MiB), or 19–32% and 12–23% counting the online phase as well. UC3 has no linear-layer triples and gains nothing. The runtime cost is in the key switches. SEAL takes 583 μ s for one Galois automorphism at $N = 8192$ on the Zen 4 hosts (1,308 μ s on Zen 3), single-threaded. Calibrated against the measured conv phase, the evaluation itself stays about the same (0.64 against 0.71 s on Zen 4), and the repacking adds 1.1 s with a merge tree ($7.2 \cdot 10^4$ key switches) or 8.2 s with traces ($5.4 \cdot 10^5$) on 32 threads; on Zen 3, 1.7 or 12.4 s. So repacking would make an inference’s preprocessing about 1.0 s slower on Zen 4 (1.8 s on Zen 3) to save 236 MiB per HE product. That breaks even at about 1.9 Gbit/s (Zen 3: 1.1 Gbit/s): it pays off on WAN links and loses on the 25 Gbit/s testbed.

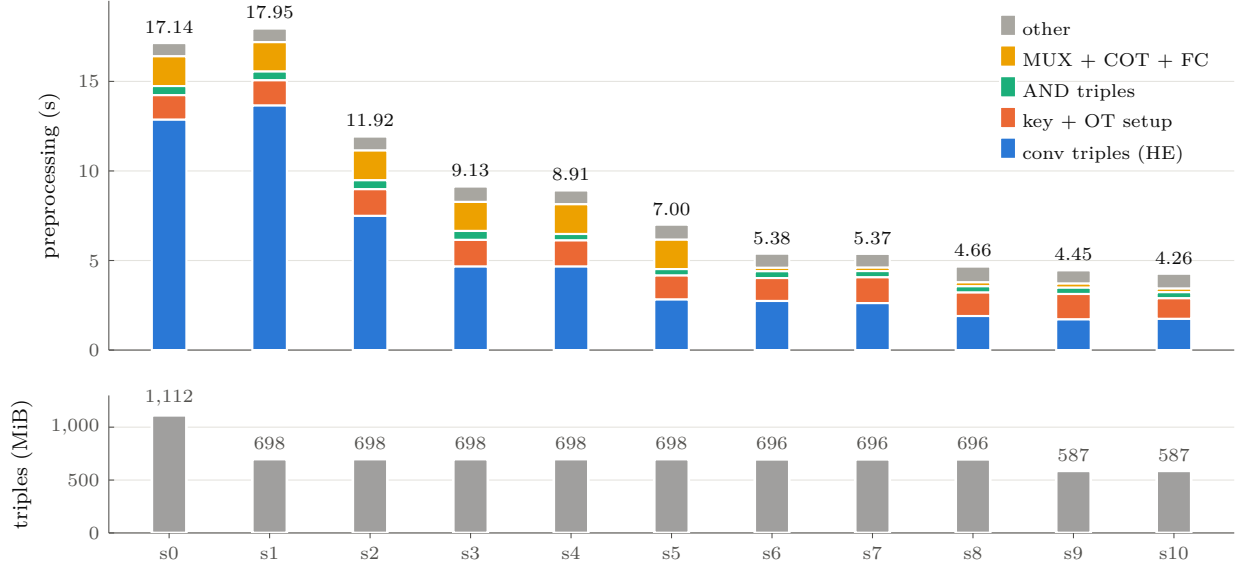


Figure 4: Preprocessing of one ImageNet image on algo / goracle, by component (median of 3 runs), and the triple traffic sent plus received by P0. s2–s6 are value-independent reruns (Section 3.2).

Output repacking, implemented. CHEETAH_CONV_REPACK=1 (round 4 of Section 9) packs with PackLWEs’ merge tree. The input polynomial holds C channel slots per position, channels fastest (C a power of two), and a weight polynomial one filter, channel c at $X^{C \cdot p - c}$ for kernel offset p ; the product then holds each output at a multiple of C , and every other coefficient mixes different channels, also where it wraps around. The evaluator merges the products of C filters of a tile level by level, $m = C/2, \dots, 1$: with the automorphism $\sigma : X \mapsto X^{N/m+1}$, which fixes X^{2mk} and negates $X^{m(2k+1)}$, the node $A + X^m B + \sigma(A - X^m B)$ doubles the used coefficients of A and B and cancels their other coefficients at the multiples of m ; after the last level filter r sits at residue r . The products are multiplied by $C^{-1} \bmod q$ first (q is odd), which the $\log_2 C$ doublings cancel exactly, so only the key-switching noise of the $C - 1$ automorphisms adds up, far below the flooding, which follows the packing (as does the mask, over all coefficients). P1 sends its Galois keys once (both parties for AB triples, 1.8 MB each). Measured on the 53 ImageNet convolutions (32 threads, all triples exact): the byte-optimal tilings take 148,622 automorphisms for 264 MiB and 6.3 s of HE; pricing each automorphism at 2 KB in the tiling search gives 26,000 for 285 MiB in 2.1 s, against 479 MiB in 0.71 s without repacking. The model’s 243 MiB assumed dense outputs; with C a power of two, the $49 \cdot 2^k$ positions of the feature maps fill only 77% of a polynomial. End to end, preprocessing traffic falls by 18–31% in UC1 and 11–24% in UC2, for 1.3–1.7 s more preprocessing on Zen 4 (break-even about 1.2 Gbit/s for UC2, 2.1 Gbit/s for UC1); the online phase is unchanged.

3.2 Engineering the HE preprocessing

Figure 4 traces ImageNet preprocessing through the ten ConvTriple changes (s1–s10); Table 4 lists what each step does.

The two machine pairs. We ran the s10 build on flare / polynize as well (Table 5). It sends exactly the same bytes, and preprocessing takes 3.31 instead of 4.26 s (22% less). Most of the gap is the convolution (1.13 against 1.74 s): the packed evaluator’s NTTs and multiply-accumulate use AVX-512 on Zen 4 and AVX2 on Zen 3. The ferret key setup and the AND triples, which are bound by memory and AES, gain only 12–14%. The online phase of this build was still dominated by delayed-ACK stalls on both pairs (medians 3.08 and 4.77 s); after the changes of Section 5 it takes 0.69–0.73 s on flare / polynize (1,820 rounds, of which P0 waited 0.15–0.16 s for data). The online steps were measured on flare / polynize only, and the PPA4 adder only there: preprocessing 10.8–11.4 s (before the shared-OT tuples) and online 1.05–1.06 s.

Packed evaluator (s1, s2). CHEETAH’s convolution encrypts one input tile per ciphertext and returns one ciphertext per output channel and tile. The packed layout of the GPU path puts several images and output channels into each ciphertext: triple traffic falls from 1112 to 698 MiB. At first this made the evaluator slower (s1), because every weight polynomial still took a full NTT. The evaluator now transforms a weight polynomial right before its only use, and it transforms 1×1 weights with NTTs of size $N/2^a$. It builds sparse polynomials from a table of powers and sums products lazily in 128 bits. Convolution time falls from 13.7 to 7.5 s.

Step	Change	Pre (s)	Conv (s)
s0	Baseline: CHEETAH’s HomConv2DSS, one output ciphertext per channel and tile	17.14	12.86
s1	Packed layout: several images and output channels per ciphertext (traffic 1112 → 698 MiB)	17.95	13.65
s2	Packed evaluator: weight polynomials transformed right before use, 1×1 weights through NTTs of size $N/2^a$, sparse polynomials from a power table, lazy 128-bit accumulation	11.92	7.49
s3	AES-CTR noise and masks instead of Blake2; ciphertexts loaded in parallel	9.13	4.66
s4	AND triples from bit-packed random OTs (64 per word)	8.91	4.66
s5	Thread pool kept across calls; multiply-accumulate blocked per prime	7.00	2.82
s6	Multiplexer and COT triples on all threads (MUX 1.08 → 0.07 s, COT 0.48 → 0.04 s)	5.38	2.74
s7	All work independent of the weights’ values (TRIPLE_ZERO=ON)	5.37	2.62
s8	Convolutions pipelined across layers (Fig. 5)	4.66	1.90
s9	Exact wire format and byte-optimal tiling (conv traffic 588 → 479 MiB)	4.45	1.71
s10	Prefetching LPN step, ferret parameters b12 (key setup 1.42 → 1.13 s)	4.26	1.74

Table 4: The HE and OT preprocessing steps (ImageNet, algoi / goracle, 32 threads). Table 5 compares the s10 build on both machine pairs.

ImageNet, one image, s10 build (median of 3 runs)	algoi / goracle (Zen 3)	flare / polynize (Zen 4)	ratio
Preprocessing (s)	4.26	3.31	1.29
key setup (ferret, HE keys)	1.15	1.01	1.13
conv triples	1.74	1.13	1.55
AND triples	0.34	0.29	1.17
multiplexer / COT / FC triples	0.07 / 0.03 / 0.10	0.08 / 0.07 / 0.08	
Triple traffic, P0 sent / received (MB)	305.5 / 281.9	305.5 / 281.9	1
Online (s), before the online changes	4.77	3.08	
Online (s), final online build (Fig. 10)	–	0.69–0.73	

Table 5: The same ImageNet build on both machine pairs. The online times of the s10 build vary with delayed-ACK stalls (flare / polynize 2.43–3.21 s over three runs).

Randomness and threads (s3–s6). Flooding noise and masks come from an AES-CTR generator (AES-NI) instead of SEAL’s Blake2; the thread pool survives across the several parallel calls of a layer; the multiplexer and COT triples run on all threads. Together these steps take preprocessing from 11.9 to 5.4 s.

Pipelining (s8). Per layer, each party idled while the other worked: at s7 P0 waited 0.82 s and P1 1.95 s in total. The pipelined protocol (Fig. 5) overlaps P1’s encryption and decryption with P0’s evaluation. It sends exactly the same bytes and cuts convolution time from 2.6 to 1.9 s; with A_KNOWN=0, where both parties encrypt, from 3.9–4.0 to 2.0 s in isolation.

Concurrent evaluation. Even pipelined, AB2 was slower than AB on CIFAR-10: 0.34–0.39 s against 0.26–0.30 s for the 23 convolutions of the isolated test, although in AB each party evaluates a full convolution under HE. Evaluating one chunk (up to 16 images) is a sequence of parallel regions: loading the inputs, the multiply-accumulate over blocks of output groups and primes, and masking, flooding and truncating the outputs. On CIFAR’s small layers these regions have fewer tasks than threads. An AB party keeps its threads busy by overlapping encryption, evaluation and decryption; the AB2 weight holder evaluated one chunk after another and sent each result before starting the next (Fig. 6). Now three evaluator threads take the chunks in order and a sender thread emits the outputs in chunk order. Chunk k draws its masks and flooding noise from the PRNG call that the serial loop gave it, so the triples are the same bits and every output hash is unchanged. Table 6 shows the effect of the number of chunks in flight. AB got the same change. End to end, the conv phase of a 10-image preprocessing halves, from 0.54–0.65 to 0.26–0.31 s.

Exact wire format (s9). SEAL serializes a seeded input ciphertext with zstd to 62.4 KB. P1 now sends only a 16-byte seed and c_0 at exactly $60 + 49 = 109$ bits per coefficient (55.8 KB). P0 expands c_1 from the seed directly in NTT form, which saves an inverse and a forward NTT per input. For outputs, the modulus switch before decryption keeps 34 bits of c_0 and 46 of c_1 ; exactly those bits travel, 20–25% less than zstd. The tile search now minimizes bytes rather than ciphertexts, since an input costs about 2.4 times an output. Convolution traffic falls 18% and all preprocessing traffic 12% (Table 7); decrypted values are bit-identical, and a check rejects outputs whose dropped bits are not zero. The 109-bit input modulus is a floor: BFV’s plaintext product carries a term of up to $\approx 2^{74}$.

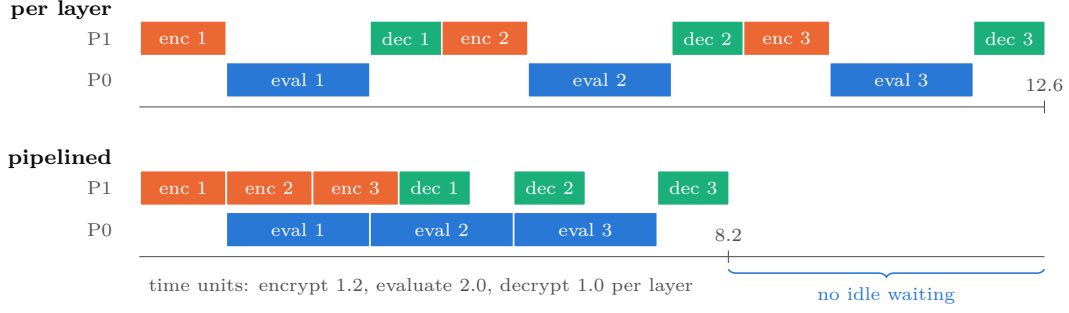


Figure 5: Pipelining the convolutions across layers (s8). Per layer, P1 waits while P0 evaluates and vice versa; pipelined, P1 encrypts the next layer’s inputs and decrypts the previous layer’s outputs while P0 evaluates. Same bytes, preprocessing 5.69 $\rightarrow$ 4.66 s.

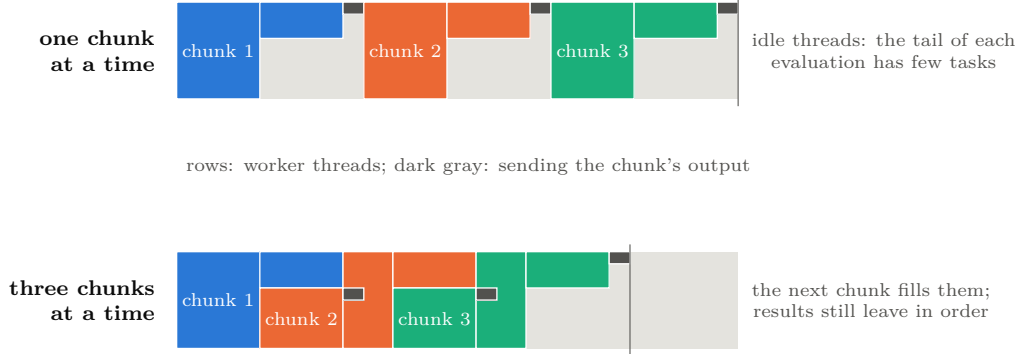


Figure 6: Why concurrent evaluation helps (schematic). The tail of each evaluation has fewer tasks than threads; with several chunks in flight, the next chunk fills the idle threads.

BatchNorm triples. Unfused BatchNorm multiplies each activation by a per-channel scale, which the slot-encoded protocol computes as an elementwise product of two vectors. The original code split each image’s elements among the 32 threads before packing: every thread sent its own nearly empty ciphertexts, one image at a time. Packing all images into full ciphertexts cut BN traffic from 2704 / 3029 to 90 / 101 MB and BN time from 6.2–7.9 to 0.7–0.8 s (CIFAR-10, 10 images). Each of the 53 BN layers was still a separate call, and one call is latency-bound (1.6–6.4 MB in 13–50 ms). `CHEETAH_BN_BATCHED` concatenates all slot-encoded layers into one product, split among the threads in whole ciphertexts: the BN phase falls from 0.75 to 0.19 s with the same traffic (Table 8).

Value independence. The benchmark’s dummy model makes every weight mask zero, and early versions of the packed evaluator skipped zero weights, so their benchmark times looked 0.6–2.9 s faster. With weights known in preprocessing, a zero-skipping evaluator would also leak the number of zero weights through its timing. All evaluators now do the same work for any weight values, and all numbers in this report are value-independent.

3.3 Conv triples and OT on a GPU

ConvTriple’s GPU path (`TRIPLE_GPU`, troy-nova) evaluates each convolution with troy’s `Conv2dHelper`, one round trip per layer, full ciphertexts on the wire. The packed CPU evaluator started from its tiling (s1) and then went further: the exact wire format, the pipeline across layers, concurrent evaluation, the start alongside the OT phase and the repacking variant. The GPU path got none of these. Now the packed pipeline itself has a GPU evaluator (`TRIPLE_GPU` builds, `CONV_GPU=0` turns it off), so all of them carry over. The device builds each weight polynomial directly in the shared memory of its NTT kernel (every coefficient, zero or a weight, computed from the weight tensor: no memset and no extra pass), transforms the inputs’ c_0 , sums the products with 128-bit accumulators (both halves of a ciphertext per thread, so each weight value is loaded once) and transforms the results back. The host keeps parsing the inputs, flooding, masking, truncation and the wire format, through pinned staging buffers. The transforms replicate SEAL’s `ntt_negacyclic_harvey` with SEAL’s own tables and slot order. So c_1 , which arrives in SEAL’s NTT form, needs no conversion, the triples are the same values as on the CPU, and the two-party check of all 53 ImageNet convolutions passes exactly.

isolated pipelined suites, flare / polynize, 32 threads chunks evaluated at once	AB2 (A_KNOWN=1)		AB (A_KNOWN=0)	
	CIFAR (s)	ImageNet (s)	CIFAR (s)	ImageNet (s)
1, sending in the evaluation loop (before)	0.34–0.39	–	0.26–0.30	1.03
1, separate sender thread	0.27–0.29	0.94		
2	0.19–0.20	0.77		
3 (default)	0.18–0.19	0.73	0.21	0.82
4	0.17–0.18	0.75		

Table 6: Conv triples with several chunks in flight (CONV_PIPE_EVALUATORS); every output checked exactly. Traffic is unchanged: CIFAR 99 MiB (AB2) and 197 MiB (AB), ImageNet 479 and 958 MiB.

MiB, P0 sent / received	before (s8)	after (s10)
Conv triples	232.6 / 355.1	234.9 / 244.1
Ferret setup	46.2 / 46.2	52.7 / 52.7
Multiplexer triples	35.4 / 35.4	35.4 / 35.4
COT triples	34.4 / 1.1	34.4 / 1.1
FC triples	0.7 / 1.2	0.7 / 1.2
Total	349.4 / 439.0 (788.4)	358.2 / 334.6 (692.8)

Table 7: Traffic of one ImageNet preprocessing (A_KNOWN=1). With A_KNOWN=0 the triple traffic falls from 659 / 626 to 551 / 518 MiB.

The conv triples alone. Table 9 compares CPU and GPU on one workstation, the only machine with a GPU, so both parties share its CPU. In isolation the GPU evaluator produces the conv triples $2.3\times$ (AB2) and $2.1\times$ (AB) faster than the packed CPU pipeline, and $5.6\text{--}7.0\times$ faster than the old GPU path, with 27% less traffic. In the whole inference that alone changes little (0.15–1.40s): on this machine preprocessing is the OT phase. With 16 one-channel OT packs per party, setting them up takes 2.0–2.5s, the A2bits builds’ Boolean addition (31 dependent COT rounds) another 4.8–5.1s, and the conv triples (4.6–6.4s on the CPU, 1.4–2.3s on the GPU) run alongside.

The OT phase on the GPU. Ferret’s extensions now run on the GPU as well, in four steps (Table 9, middle). (1) The LPN step: ten AES-indexed gathers per output, from a table of $k = 238,000$ blocks that stays in the GPU’s L2 cache; each lane keeps its own copy of the AES T-table in shared memory, so the table lookups never conflict. (2) MPCOT: all 2,507 GGM trees of an extension are built level by level, the sender’s from emp’s seeds, the receiver’s from its OT messages, with the per-level XOR sums reduced across warps. The LPN step then runs on the leaves in place, and one copy returns the outputs. Each channel’s tree messages leave as one message, the same bytes as emp’s 2,507 separate flushes. (3) The outputs stay on the GPU. In the ImageNet runs, 97% of all COTs go to random OTs with 1-bit messages (or k bit planes), which hash each COT with emp’s MITCCRH (a fresh AES key per OT) and keep one bit or a few. The GPU hashes them in place, scheduling each key on the fly, and returns only the bit planes: two to six bits per OT cross PCIe instead of 128, and the CPU does no hashing. The other consumers get host copies of their range on demand. (4) With extensions this cheap, setting up the packs dominated, so a GPU build uses 4 packs of 4 channels instead of 16 of 1: setup 0.8–1.2s, preprocessing 0.6–1.3s lower. Every step yields emp’s values bit for bit. Check modes compare every tree, output and hashed bit with emp’s CPU code. The Boolean triples pass the two-party tuple test, and seeded CIFAR-10 ResNet50 runs give the CPU build’s output hashes (with the same pack layout). The bytes on the wire do not change, so either party may run either path. Each ferret instance keeps its 164 MiB of outputs on the device; with both parties on one 20 GB GPU there is room.

End to end. With conv triples and OT on the GPU, preprocessing is $2.2\times$ faster than on the CPU in the A2bits builds and $1.7\text{--}1.8\times$ in the reshared ones (Table 9, bottom). The online phase measured 0.02–0.07s higher with the GPU OT path. Pinning, a 5s pause before the online phase, OT work during the online phase (there is none) and memory pressure are ruled out; on this shared machine the gap is still unexplained. The online conv layers spend their 0.2s on the per-output mask work and the exchange with the other party, not on the products: moving GEMM_FAST’s product to the GPU (GEMM_FAST_GPU, exact, checked against the CPU on sampled entries) leaves the UC2 online phase at 0.46s. HPMPC’s own CUTLASS convolution (USE_CUDA_GEMM=2) bypasses the per-output mask step and with it the A2B bake. What remains of preprocessing on the CPU is the packs’ setup (base OTs, IKNP and each pack’s first extension), the A2bits builds’ Boolean addition (latency-bound COT rounds), the 3% of COTs whose consumers need the blocks themselves, and the HE work around the GPU evaluator (encryption, flooding, decryption).

CIFAR-10, 10 images	traffic P0 sent / recv (MB)	BN phase (s)	preprocessing (s)
threads split each image	2704 / 3029	6.2–7.9	8.2–10.0
full ciphertexts	90 / 101	0.7–0.8	3.0–3.2
all 53 layers in one product	88 / 99	0.19	1.94 (RCA)

Table 8: BatchNorm triples with unfused BN. In multi-batch the per-process BN phase falls from 1.16 to 0.85 s (3 threads per process, compute-bound).

conv triples, ImageNet, 53 convolutions	AB2 (s)	AB (s)	traffic AB2 / AB (MiB)
old GPU path (troy, per layer)	5.26	10.22	655 / 1,310
CPU, packed and pipelined	2.18	3.02	479 / 958
GPU evaluator in the packed pipeline	0.94	1.45	479 / 958

ferret on the GPU: one OT pack, $2 \cdot 10^8$ Boolean triples	time (s)
CPU (emp-ot)	38.5–38.6
LPN step on the GPU	22.3–24.2
+ MPCOT’s trees on the GPU, LPN on the leaves in place	17.6–20.0
+ outputs copied straight into ferret’s buffer, one message per channel	14.4–15.1
+ outputs stay on the GPU, ROT bits hashed there	6.2–6.6

ImageNet, final flags	preprocessing (s)			online (s)		
build	CPU	GPU conv	GPU all	CPU	GPU conv	GPU all
UC1, A2bits, RCA	13.66	12.26	6.21	0.65	0.66	0.67
UC1, reshared, RCA	7.62	6.97	4.32	0.69	0.68	0.76
UC2, A2bits, RCA	12.56	11.86	5.73	0.46	0.48	0.53
UC2, reshared, RCA	6.73	6.58	3.89	0.51	0.51	0.56

Table 9: CPU against GPU on one workstation (RTX 4000 Ada, two Xeon Silver 4410Y with 24 cores, both parties on it, loopback). Top: the conv triples in isolation (16 threads per party, median of three runs), exact in every run. Middle: ferret’s extensions step by step (ConvTriple’s tuple test, both parties, two runs each). Bottom: the whole inference (hpmc 92a2c0e, ConvTriple 111e7be; 16 HE/OT threads and 11 pool workers per party, as the configurations’ 32 and 24 oversubscribe a machine both parties share; median of three runs). GPU conv: the conv triples on the GPU evaluator (LPN_GPU=0); GPU all: also ferret’s extensions and the ROT bits.

4 Boolean material: OT preprocessing

4.1 Boolean correlations: protocol and security

All Boolean material is XOR-shared: $v = v^A \oplus v^B$ between the two OT roles A and B (P0 and P1, or reversed in the second direction). This subsection states the protocols of `ot_proto.hpp` and `silent_ot.h`.

Random OT with k -bit messages. The functionality $\mathcal{F}_{\text{ROT}}^k$ gives the sender two uniform strings $r_0, r_1 \in \{0, 1\}^k$ and the receiver a uniform choice bit s and r_s . Ferret silent OT (LPN parameters b12) produces correlated OTs: the sender gets $K_i \in \{0, 1\}^{128}$ and a global Δ with $\text{lsb}(\Delta) = 1$, the receiver $M_i = K_i \oplus s_i \Delta$ with $s_i = \text{lsb}(M_i)$. A tweakable correlation-robust hash H (emp’s MITCCRH, with a fresh key per call) turns them into random OTs, $r_b = H(i, K_i \oplus b\Delta)|_k$ for the sender and $r_{s_i} = H(i, M_i)|_k$ for the receiver. One hash yields 128 bits, of which `send/recv_rot_bitplanes` keep $k \leq 64$ as k independent message bits and store them as k bit planes, eight OTs per byte. By correlation robustness, r_{1-s_i} is pseudorandom to the receiver: a k -bit random OT costs one COT and one hash per message, the same as a 1-bit one.

Products with a common factor. Let $x_1, \dots, x_k$ and y be shared bits; the goal is shares of $z_p = x_p y$ for all p . Expanding, $z_p = x_p^A y^A \oplus x_p^B y^B \oplus x_p^A y^B \oplus x_p^B y^A$: two local terms and two cross terms, one in each direction. Figure 7 computes the cross term $x_p^A y^B$ for all p from a single $\mathcal{F}_{\text{ROT}}^k$ with A as sender. The other direction is symmetric, with B as sender.

Lemma 2 (correctness). $u_p \oplus v_p = x_p^A y^B$. *Proof.* If $y^B = 0$, then $e = s$ and $u_p \oplus v_p = r_{s,p} \oplus r_{s,p} = 0$. If $y^B = 1$, then $e = 1 - s$ and $u_p \oplus v_p = r_{1-s,p} \oplus r_{s,p} \oplus x_p^A \oplus r_{0,p} \oplus r_{1,p} = x_p^A$. With the fresh factor, $y^B = s$ and $e = 0$, and the same two cases hold. $\square$

Theorem 2 (security). In the $\mathcal{F}_{\text{ROT}}^k$ -hybrid model, Π_{outer} securely computes shares of $\{x_p y\}_p$ against a semi-honest adversary; with ferret and a correlation-robust hash realizing $\mathcal{F}_{\text{ROT}}^k$ (under the LPN assumption),

Protocol Π_{outer} (one element, direction $A \rightarrow B$; <code>cot_outer_multiply</code>) <i>Inputs:</i> A: $x_1^A, \dots, x_k^A$. B: y^B, or nothing if B's share of y is to be fresh. <i>Outputs:</i> A: $u_1, \dots, u_k$; B: $v_1, \dots, v_k$ and, if fresh, y^B; with $u_p \oplus v_p = x_p^A y^B$. 1. $\mathcal{F}_{\text{ROT}}^k$: A gets $r_0, r_1 \in \{0, 1\}^k$, B gets s and r_s. 2. B corrects its choice. If y^B is given, B sends $e = s \oplus y^B$ (1 bit). If y^B is to be fresh, B sets $y^B := s$ and sends nothing; A uses $e = 0$. 3. A masks its k values with the sum of both messages: sends $m_p = x_p^A \oplus r_{0,p} \oplus r_{1,p}$ (k bits). 4. A outputs $u_p = r_{e,p}$. B outputs $v_p = r_{s,p} \oplus (y^B \wedge m_p)$. Both directions run in parallel, and each party adds its local term $x_p^P y^P$: shares of $x_p y$ for all p from two random OTs and, per party, k bits (plus one correction bit unless y is fresh).

Figure 7: Products $x_p y$ with a common factor from one random OT per direction.

	random OTs		bits sent per party, products	
	one per product	Π_{outer}	one per product	Π_{outer}
3-tuple (3 products with c)	8	4	6	3
4-tuple (9 products of $\{a, b, ab\} \times \{c, d, cd\}$)	22	10	18	12

Table 10: Cost of one Beaver tuple, AND triples included in the OT counts. “One per product”: a separate correlated-OT multiplication for each product, as before this work.

so does the full protocol. *Proof sketch. Corrupted A (sender).* It receives $e = s \oplus y^B$; s is uniform and independent of A's view, so e is a uniform bit and the simulator sends a uniform bit (nothing, if y^B is fresh). A's output $u_p = r_{e,p}$ is uniform, and B's output completes it to $x_p^A y^B$. *Corrupted B (receiver).* It receives $m_p = x_p^A \oplus r_{0,p} \oplus r_{1,p} = x_p^A \oplus r_{s,p} \oplus r_{1-s,p}$; since $r_{1-s,p}$ is uniform and unknown to B, m_p is uniform, and the simulator sends k uniform bits. With the fresh factor, B's share $y^B = s$ is a uniform bit independent of A's view, exactly as a freshly sampled share would be. All messages are simulated independently of the other party's inputs, and every output share is uniform subject to the XOR relation. The k bits of one hash are independent pseudorandom bits under correlation robustness, so sharing one OT among k products does not correlate them. $\square$

Beaver tuples. A 3-tuple is (a, b, c) with shares of all products ab, ac, bc, abc ; a 4-tuple has (a, b, c, d) and all 11 products of two or more of them. PPA4's 4-input AND gates consume them online in one round.

- **3-tuple:** an AND triple (a, b, ab) from two random OTs (`send/recv_rot_bits`), then one Π_{outer} per direction with $x = (a, b, ab)$ and the fresh $y = c$: ac, bc and abc . Total: 4 random OTs and, for the products, 3 bits per party.
- **4-tuple:** AND triples (a, b, ab) and (c, d, cd) (4 random OTs), then Π_{outer} with $x = (a, b, ab)$ for each $y \in \{c, d, cd\}$: the nine products $\{a, b, ab\} \times \{c, d, cd\}$ (6 random OTs). cd is not fresh, so these calls send the correction bit. Total: 10 random OTs and, for the products, 12 bits per party.
- **Party-local tuples** (`RESHARE_OPT_SIM`). P0 holds b and P1 holds c in full, the other shares being zero. The same protocol runs with the zero share as a given input: only P0's zero share of c needs a correction bit.

Where the protocols are used. AND triples feed the RCA and PPA adders, 3- and 4-tuples the PPA4 adder, COT triples the bit injection and multiplexer triples the ReLU's final selection. The A2B bake of the a2b family (`A2B_CONV_BAKE`) computes Boolean shares of $-(\ell_0 + \ell_1)$ for the committed conv output masks, where each party knows its own ℓ_i in full: a ripple-carry addition whose 31 carry levels are one multiplication each, run over all values of the network at once (ImageNet: 9.0 million values per level). Every OT consumer draws from the same OT packs (one per HE/OT thread, two ferret instances each, one per direction), which are sized from the total demand of the preprocessing before their first extension (`ot_demand_hint`) and extend in parallel.

4.2 Engineering the OT preprocessing

The ReLUs consume AND triples, Beaver 3- and 4-tuples (PPA4), COT triples (bit injection) and multiplexer triples. All come from ferret silent OT, 32 OT packs of two ferret instances each.

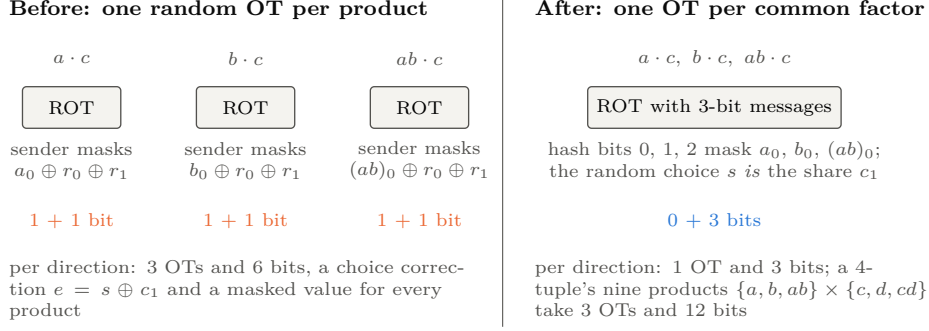


Figure 8: Products with a common factor share one random OT. Before, each product $x \cdot c$ took one random OT per direction plus two sent bits. Now one OT per direction carries all k products in its k message bits, and a party whose share of c is fresh takes it from the OT’s random choice bit.

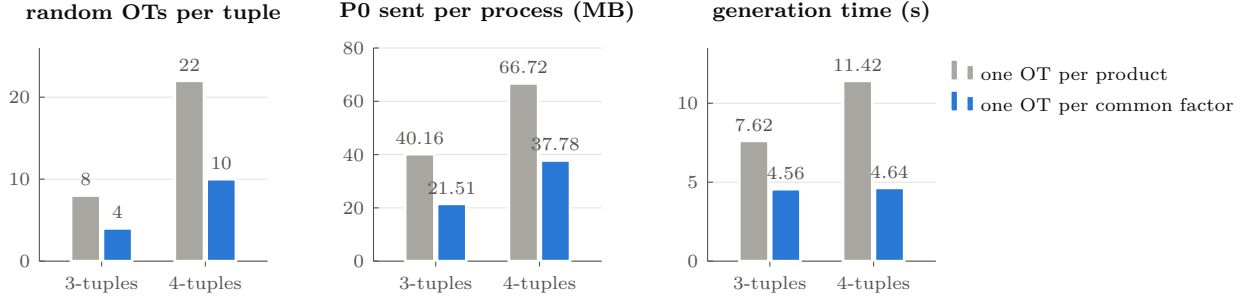


Figure 9: Beaver tuple generation per process in a multi-batch PPA4 run (flare / polynize, 24 processes).

Ferret parameters and the LPN step. With 32 instances at once, ferret’s LPN step is memory-bound on its 7.2 MB tables. Computing the AES of the next four groups ahead and prefetching their table rows cuts the step from 0.34 to 0.28 s with bit-identical outputs. Parameter set b12 has a 3.8 MB table that stays in cache, for 6.5 MB more setup traffic per direction. Key setup falls from 1.42 to 1.13 s. VAES did not help: the LPN step is memory-bound and MITCCRH is bound by its AES key schedule.

Demand-sized OT packs. The first OT consumer used to size the packs, and later consumers extended them in small steps. hpmc now passes the total OT demand of the whole preprocessing to ConvTriple (ot_demand_hint) before the first generation. On CIFAR-10 the OT-pack phase falls from 0.91 to 0.47 s, and PPA4, reshare and a2b get enough capacity for their tuples.

TCP_NODELAY everywhere. The ConvTriple channels enabled TCP_NODELAY only in WAN mode. The COT multiply behind the a2b bake flushes small messages 31 times in sequence, and each flush waited for a delayed ACK. One call took 170 ms; with Nagle off it takes 11 ms. PPA a2b single-batch preprocessing falls from 8.4 to 3.5 s, PPA4 reshare+sim from 8.9 to 5.0 s. The same fix on hpmc’s own sockets takes the ImageNet online phase from 2.4–4.3 to 1.9–2.2 s (Section 5).

Bit-packed random OTs. Random OTs with one-bit messages are hashed and reduced to bits in cache-sized chunks and stored eight per byte (send/recv_rot_bits), where the generic interface kept two 16-byte blocks per OT. AND triples take 0.36 instead of 0.50 s, and the COT multiply reads the same bits with far less memory traffic.

Beaver 3- and 4-tuples with shared OTs. Before this work, a 3-tuple was an AND triple plus three independent correlated-OT multiplications $a \cdot c$, $b \cdot c$ and $ab \cdot c$, each with its own random OT per direction. Π_{outer} (Section 4.1) computes all products with the common factor c from one random OT per direction, whose k message bits (send/recv_rot_bitplanes) cost one hash, and takes the fresh c from the OT choices (Fig. 8). A 3-tuple now needs 4 random OTs instead of 8 and a 4-tuple 10 instead of 22 (Table 10). In a multi-batch PPA4 run the tuple phases fall from 19.0 to 9.2 s per process and their traffic by 43–46% (Fig. 9). Unit tests check all products; ReLU exactness tests report 0 wrong outputs.

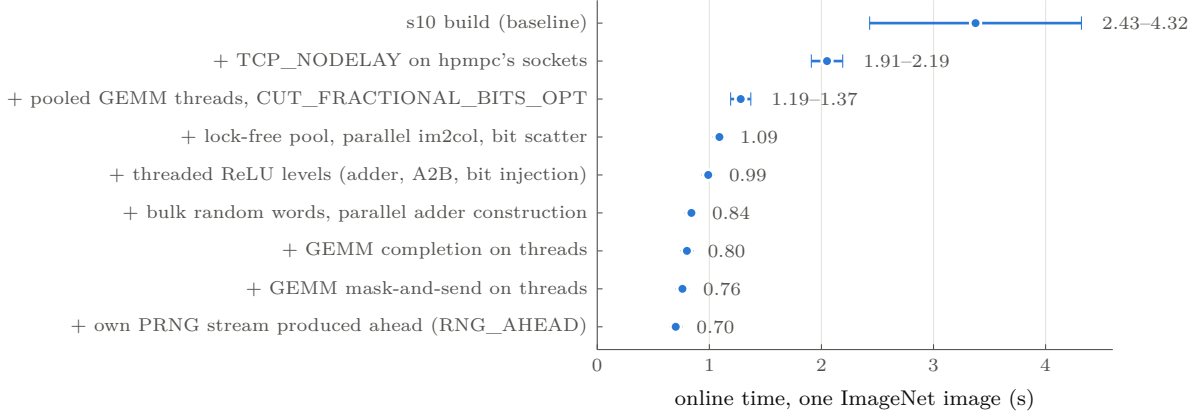


Figure 10: Online phase of one ImageNet image on flare / polynize (ranges over runs where they varied). Output hashes are unchanged by every step.

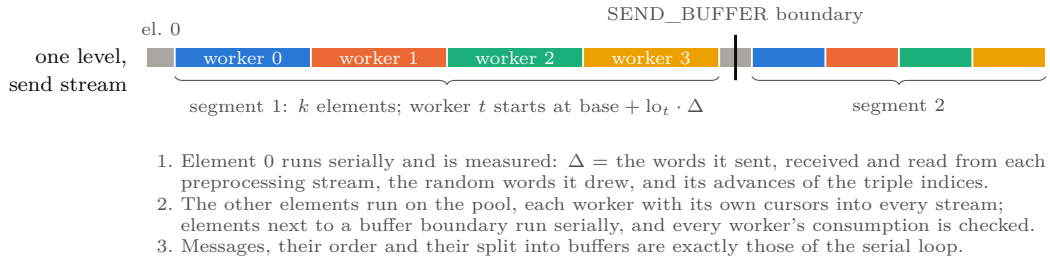


Figure 11: `stream_parallel_for`: element-parallel circuit levels at batch size one. One element is measured; the level's remaining elements run on the worker pool with per-worker cursors into every stream, split at buffer boundaries.

5 Online phase

On a 0.3ms LAN, the online phase is bound by round latency and per-round work, not by bandwidth. Activations send about 180 MB per ImageNet inference, 0.06 s at 25 Gbit/s. Figure 10 shows the steps for one ImageNet image.

Nagle, buffers and GEMM threads. With Nagle's algorithm, a round's last small message waited for a delayed ACK. ReLU layers were bimodal (6 or 46 ms), and the online phase varied between 2.4 and 4.3 s. With TCP_NODELAY it takes 1.9–2.2 s. Send and receive buffers of 100 000 words mean fewer messages and thread hand-offs per round. The threaded GEMM first sent its outputs in linear order while the completion visits 64×64 tiles; CIFAR-10 accuracy was 0% until both used the tile order. Its threads now live in a lock-free pool.

Element-parallel levels at batch size one. HPMPc vectorizes only across independent inputs, so a single image evaluated every level of a ReLU (adder steps, A2B, bit injection) element by element on one thread. Within a level, every element runs the same code and consumes the same number of words from each stream: sends, receives, preprocessed values, its own random words and the indices into the triple buffers. `stream_parallel_for` measures this consumption on the first element and runs the rest on the worker pool, each worker with cursors at exactly the stream positions the serial loop would use (Fig. 11). Segments end at send/receive buffer boundaries, and a misuse check verifies every worker's consumption. The messages, their order and all values are those of the serial loop. With bulk random words, parallel adder construction and the GEMM mask-and-send and completion on the same pool, the ImageNet ReLU time falls from 630 to 300 ms and the online phase from 1.09 to 0.70 s. `RNG_AHEAD` moves the party's own sequential AES stream to a background thread.

Rounds per ReLU. What remains is round latency: 1,752 rounds for CIFAR-10 with RCA against 776 with PPA and 625 with PPA4 (Table 11). `CUT_FRACTIONAL_BITS_OPT` removes the fractional bits from the MSB adder by substituting identity elements and public constants. RCA loses 245 of its 1,997 rounds (12%), and ReLU traffic falls 8–11% for every adder. Preprocessing and accuracy do not change measurably (1,134 vs. 1,138 of 1,752 evaluations correct with and without CUT).

single batch, unfused BN	rounds		ReLU traffic (MB)		online (s)	
	CUT	no CUT	CUT	no CUT	CUT	no CUT
RCA	1,752	1,997	20.8	23.3	0.52	0.48
PPA	776	789	32.6	36.8	0.44	0.49
PPA4	625	625	20.1	21.8	0.42	0.42

Table 11: Online rounds, P0’s ReLU traffic and online time for 10 CIFAR-10 images on flare / polynize, with and without CUT_FRACTIONAL_BITS_OPT. The time differences are within run-to-run noise.

multi-batch, 192 images, flare / polynize	preprocessing (s)		conv phase per process (s)		P0 sent (MB) before → lanes
	before	lanes	before	lanes	
RCA, unfused	13.7	8.9	8.64	3.87	8,884 → 4,459
RCA, fused	12.5	8.0	8.43	3.86	7,188 → 2,763
PPA, unfused	16.9	12.0	8.73	3.88	9,022 → 4,597
PPA4, unfused	22.6	16.7	8.67	3.82	10,238 → 5,813

Table 12: Conv triples of all lanes as one convolution (CHEETAH_CONV_LANES), plain family, 3 HE/OT threads per process.

6 Multi-batch

With 24 processes of 256-bit lanes, preprocessing dominates. With more than one HE/OT thread per process, the processes shared connections: channel i used port $base + i \cdot PROCESS_NUM$ while process bases stepped by 3, which caused hangs and wrong triples. With a stride of $3 \cdot PROCESS_NUM$, preprocessing falls from 25.0 to 14.0s with three threads per process (Table 13). One extra online thread per process cuts the online phase from 0.81 to 0.67s with an identical output hash.

Lanes in one convolution. Each 256-bit register holds eight images (lanes), and the conv triples were generated as one single-image convolution per lane and layer, each with its own copy of the weight masks: 424 small, unpipelined convolutions per process, with ciphertexts that are mostly empty. The model owner’s mask of a weight is $-w$ (SHARE_PREP), and every lane’s image uses the same weights, so all lanes have the same weight masks by construction. The eight triples of a layer are therefore one convolution over eight images with one set of weights. CHEETAH_CONV_LANES generates them that way and passes all layers to one pipelined call. A debug check confirmed that no lane’s masks differ in any layer or process. With three HE/OT threads per process the conv phase falls from 8.4–8.7 to 3.8–3.9s and P0’s preprocessing traffic by 50–62% (Table 12); four threads, which enable the pipeline, gain nothing more. With the AdamW model the accuracy is unchanged (PPA 121 of 192 against 123 without, RCA 131, PPA4 127). The premise does not depend on A_KNOWN: input sharing gives the model owner the share $-w$ and the other party 0 in every lane either way, and A_KNOWN=0 only turns the triple into an AB triple, for which both parties encrypt their shares. The lane batching therefore also applies to A_KNOWN=0. In a local test (one process, 8 lanes of 4 images, 4 HE threads, AdamW model) the lane check stays silent for both parties, the conv phase falls from 12.9 to 7.2s and the conv traffic from 1036 to 540MB per direction, and 22 against 23 of 32 images are classified correctly. On flare / polynize (24 processes, 192 images) the conv phase per process falls from 9.7–10.0 to 4.1–4.4s and preprocessing by a factor of 1.28–1.67 over the 30 variants (Section 8.1).

7 Correctness and reproducibility

Fast code is only useful if its outputs are right and repeatable. We made every run bit-reproducible, compared each threaded build against a serial one and bisected every configuration that lost accuracy over the added optimizations; this exposed the following bugs.

- **Nondeterministic OT.** COT and multiplexer triple shares are the ferret outputs, and hmpc truncates shares locally, so their per-run randomness changed the outputs. ConvTriple’s PRGs are now seeded per party, stage, call and task, and a scoped override seeds emp’s default PRGs per OT pack and worker (DetSeedScope).
- **Worker race in P0’s A2B.** For the reshare+sim skip with PPA4, P0 chooses each group’s slice masks from the Beaver 3-tuples that the group’s adder will consume, found through a count of groups prepared so far. On the worker pool that count was one shared global. Threaded PPA4 reshare+sim with fused

HE/OT threads	preprocessing (s)	conv (s)	AND triples (s)	BN (s)	online (s)
1	25.0	16.5	3.6	2.1	0.63
2	17.0	10.2	3.3	1.4	0.68
3	14.0	8.8	3.0	1.2	0.75
4	14.0	7.8	3.8	1.0	0.69

Table 13: Multi-batch CIFAR-10 (24 processes, 192 images, RCA, unfused BN) on flare / polynize, by HE/OT threads per process, after the port-stride fix.

BN gave a different output every run. The count is now a per-worker stream index. The race stayed hidden whenever the pool threads woke one after another.

- **Hangs.** A lost wake-up at connection start, where flag and broadcast were not under the waiters’ mutex, and a stale listener into whose backlog the next phase’s peer connected.
- **Multi-batch a2b.** The A2B mask bake derived the conv mask by casting transposed words to lanes, which holds only for 32-bit lanes; with 256-bit registers the a2b families classified 13–29 of 192 images. The mask is now computed value by value with the A2B’s own inverse packing, which leaves single-batch hashes unchanged, and the multi-batch a2b families classify 99–132 of 192 images, like the plain family.
- **Threaded GEMM with A_KNOWN=0 or public weights.** The threaded GEMM had diverged from the serial one. With A_KNOWN=0 it sent the outputs without the conv triple’s share and left the triple cursor behind, so every threaded build classified at chance (1 of 10). With public weights and TRUNC_DELAYED=0 it truncated the first layer with the regular truncation while the completion expected the data owner’s exact one, and both parties waited forever in preprocessing. It now follows the serial path.
- **BatchNorm triples with A_KNOWN=0.** AB triples multiply the full masks, and the BN product’s operand is $x - \mu$, whose mask includes the model owner’s mask of μ . The triple was set up over the masks of x , which left out $\lambda_s \lambda_\mu = s\mu$: every output was $s \cdot x + \beta$ (unit test failed, ResNet50 unfused 0 of 10). With A_KNOWN=1 only the data owner’s mask share enters, and μ does not change it.
- **Public weights with unfused BN and delayed truncation.** With public weights the first convolution’s output is still the data owner’s sharing (the other party’s mask is zero). With TRUNC_DELAYED=1, with or without BIT_INJECTION_TRUNC_SIM, the BN truncated it with the SecureML local truncation, which wraps for positive values (0–1 of 10 correct). The owner now truncates it exactly and re-masks, in the same single message.
- **Test harness.** The ReLU tests fed unbaked inputs to the reshare+sim skip (35% wrong); the harness now declares its inputs unbaked.

After these fixes, all 60 threaded variants (30 single-batch, 30 multi-batch) reproduced their serial build’s output hash on flare / polynize (round 4), and repeated runs are bit-identical. The concurrent conv-triple evaluation keeps every single-batch hash of round 4. The lane batching gives the multi-batch triples different random shares, so those hashes changed; its correctness rests on the lane check and on the accuracy (Section 6 and Fig. 15). Hashes differ between the machine pairs, because the PRNG draws 512-bit AES blocks with VAES-512 on Zen 4 and 256-bit blocks on Zen 3.

8 Evaluation of all variants

Figure 12 shows preprocessing and online time for every variant on both machine pairs.

Single batch. On flare / polynize, preprocessing takes 1.3–1.8 s with RCA, 1.9–2.2 s with PPA and 2.1–2.5 s with PPA4 when BN is fused, and 1.6–2.0, 2.2–2.4 and 2.3–2.7 s unfused; the a2b families sit at the upper end. The online phase takes 0.35–0.50 s. Because it is round-latency-bound, PPA and PPA4 are slightly faster online than RCA despite more AND gates. The fastest configuration in total is RCA with fused BN (1.32 s + 0.39 s).

Multi-batch. 192 images take 7.8–11.7 s (RCA), 11.0–15.0 s (PPA) and 13.9–19.5 s (PPA4) of preprocessing and 0.54–0.81 s online. Per image that is 41–101 ms of preprocessing, against 132–267 ms in single batch. The a2b+AKTE family is the fastest PPA4 variant here (13.9–14.8 s): its adder needs fewer tuples.

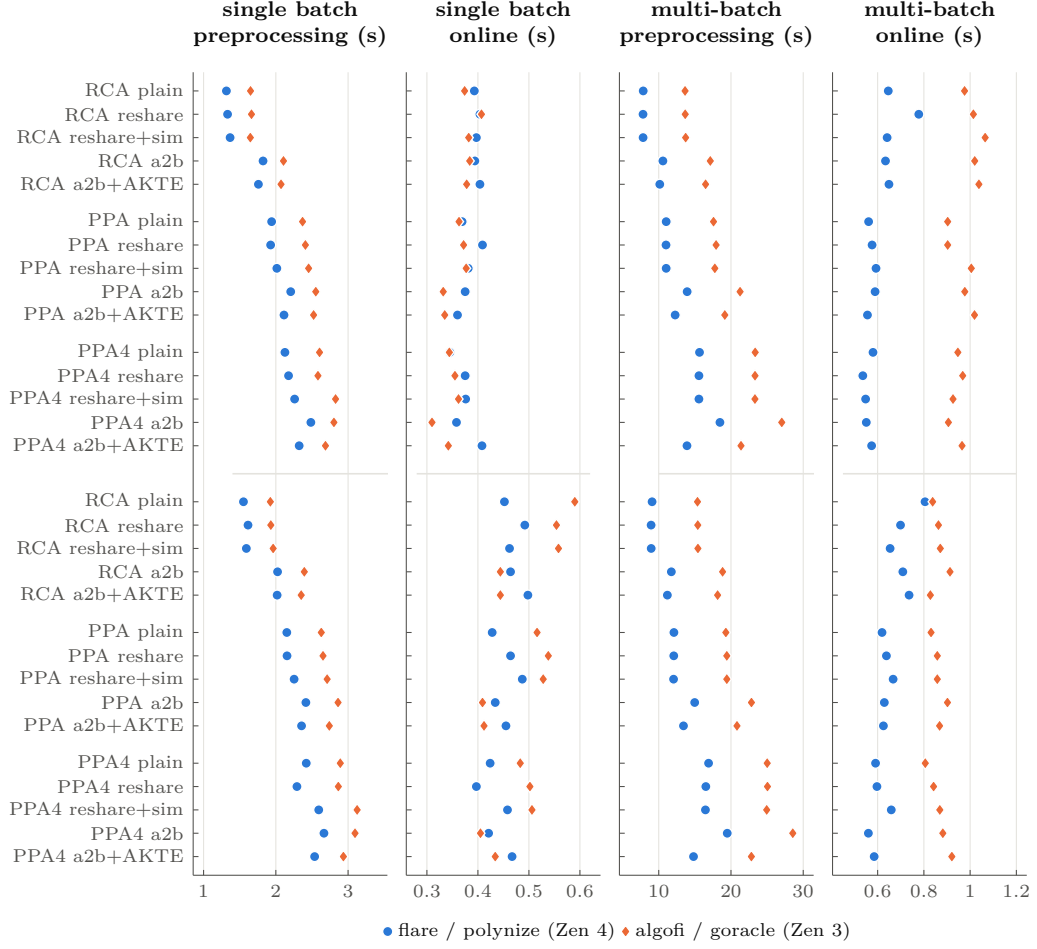


Figure 12: All 30 variants, CIFAR-10 ResNet50 (final builds). Upper block: fused BN; lower block: unfused BN. Single batch: 10 images, median of two runs. Multi-batch: 192 images, 24 processes. Tables with every value: [docs/VARIANT_PERFORMANCE.md](#).

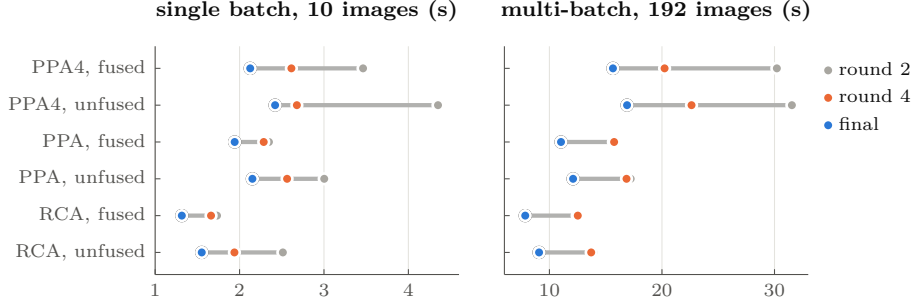


Figure 13: Preprocessing of the plain family in round 2, round 4 (after the shared-OT tuples and the batched BN triples) and with the final code (concurrent conv-triple evaluation; multi-batch: lane batching), flare / polynize.

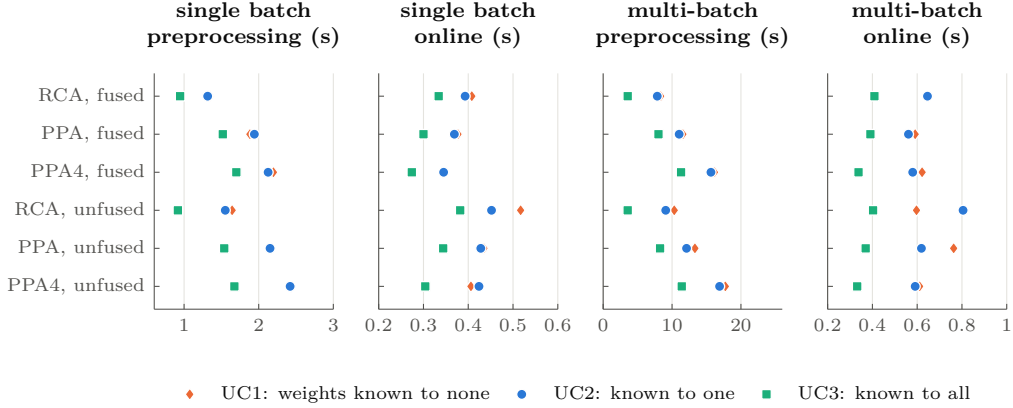


Figure 14: The three use cases, plain family, flare / polynize: UC1 (weights known to none, $A_KNOWN=0$), UC2 (known to the model owner, $A_KNOWN=1$, default) and UC3 (public weights, no conv triples). Final builds; multi-batch with lane-batched conv triples in UC1 and UC2.

Machines. In single batch, algo / goracle needs $1.13\text{--}1.25\times$ the preprocessing time of flare / polynize and $0.84\text{--}1.30\times$ the online time. Multi-batch is compute-bound, and there Zen 3 needs $1.46\text{--}1.75\times$ as long for preprocessing and $1.04\text{--}1.84\times$ online: it lacks AVX-512 for the HE NTTs and the GEMM. The lane batching gains less there (conv phase per process $14.8 \rightarrow 8.0$ s against $8.4 \rightarrow 3.8$ s; preprocessing 20–35% less against 19–38%). ImageNet shows the same pattern (Table 5).

The last optimizations. Figure 13 separates the steps. The shared-OT tuples and the batched BN triples (round 2 to round 4) help PPA4 most: single batch $4.35 \rightarrow 2.68$ s unfused and $3.46 \rightarrow 2.61$ s fused, multi-batch $31.5 \rightarrow 22.6$ s and $30.2 \rightarrow 20.2$ s; unfused RCA gains $2.51 \rightarrow 1.94$ s from the batched BN. The concurrent conv-triple evaluation then saves $0.26\text{--}0.55$ s in every single-batch variant (10–26%), and the lane batching $4.5\text{--}6.3$ s in every multi-batch variant (19–38%). From round 2 to the final code, preprocessing of the plain family falls by 17–44% in single batch and by 30–48% in multi-batch.

8.1 Weights known to none (UC1) and to all (UC3)

Besides UC2, the default, two use cases change who holds the weights. In UC1 ($A_KNOWN=0$) both parties hold shares of the weight masks as well as of the activation masks, so the conv, FC and BN triples are AB triples for which both parties encrypt. In UC3 (public weights, $PUBLIC_WEIGHTS=1$) the linear layers multiply locally and need no triples at all. We run public weights with $TRUNC_DELAYED=1$ and $BIT_INJECTION_TRUNC_SIM=1$: each linear layer’s truncation is deferred and folded into the next ReLU’s bit injection instead of costing a round of its own. Figure 14 compares the three use cases for the plain family.

UC3: public weights. Only the ReLU material remains to be preprocessed: across all 30 variants, preprocessing takes $0.92\text{--}2.03$ s per 10 images and $3.5\text{--}14.3$ s per 192 images, 16–41% and 22–61% less than the same variant in UC2 in the final builds. The online phase takes $0.27\text{--}0.41$ s and $0.33\text{--}0.42$ s (9–33% and 30–50% less): convolutions send nothing, and the truncation shares the bit injection’s message. With the AdamW model, the UC3 variants classify 63–73 of 100 images in single batch and 115–131 of 192 (60–68%) in

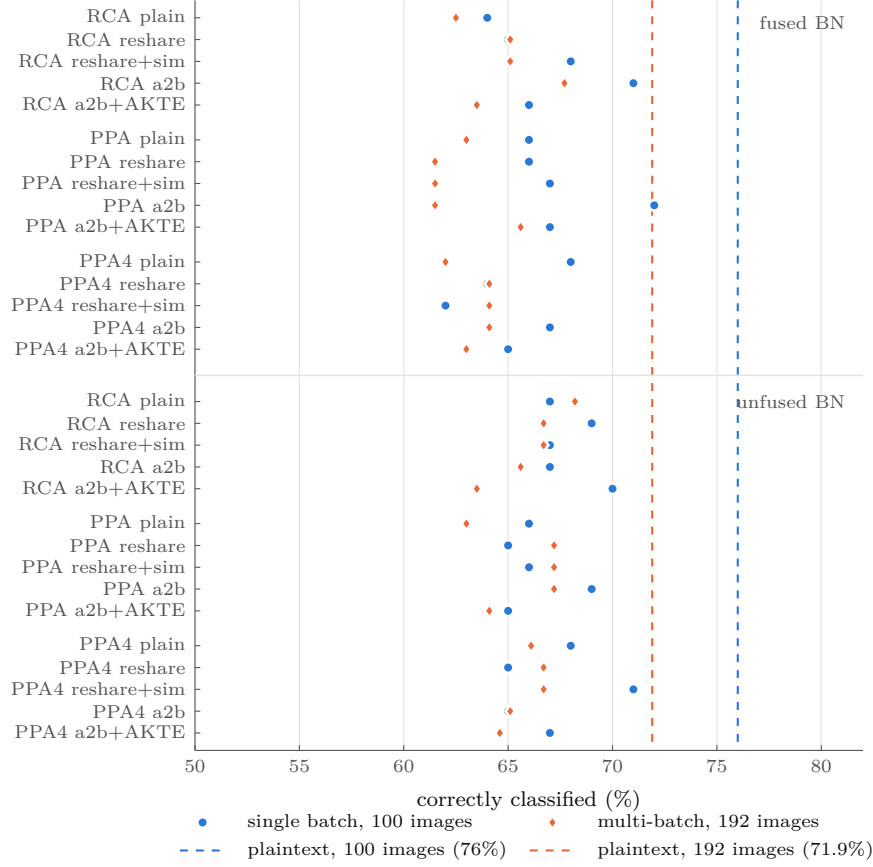


Figure 15: Accuracy with the AdamW-trained ResNet50, standard probabilistic truncation, final builds on flare / polynize: 100 images in single batch, 192 in multi-batch (lane-batched conv triples).

multi-batch. Unfused BatchNorm needed the first-layer truncation fix of Section 7; before it, those variants classified 0 of 10 images.

UC1: weights known to none. With $A_KNOWN=0$ the linear-layer triples are AB triples, and the preprocessing traffic grows by a factor of 1.4–1.9 (RCA, unfused BN, 10 images: 229 to 428 MB). In single batch, preprocessing takes 1.34–2.74s over the 30 variants and the online phase 0.34–0.52s. Since the concurrent evaluation (Section 3), $A_KNOWN=1$ is faster in both HE phases of every variant (conv 0.27–0.33 against 0.30–0.39s, BN 0.20–0.24 against 0.23–0.26s); the totals differ by -0.07 to $+0.13$ s, within the noise of the OT phases. In multi-batch, with the lane batching (Section 6), UC1 takes 8.3–20.6s over the 30 variants (before it, 13.8–26.4s), 0.4–1.3s (2–14%) more than UC2; the online phase takes 0.56–0.78s. With the AdamW model UC1 classifies 64–72 of 100 images in single batch and 116–133 of 192 in multi-batch. Its accuracy first looked like chance level: a bisection over the added optimizations found the threaded GEMM (with the extra threads off, 5 of 10 correct) and, for unfused BN, the BatchNorm triples, both described in Section 7.

8.2 Final accuracy

Every variant stays above 60% (Fig. 15 and Table 14). The spread between variants (62–72 of 100, 118–131 of 192) is truncation noise: the variants draw different amounts of randomness, so different values wrap. The adders are exact, with 0 of 32,768 random ReLU inputs wrong for each adder. Fused and unfused BN are equally accurate with this model. With the older `Cifar_adam_001` model, probabilistic truncation in the 32-bit ring cost 10–17 points (Trio 56% against 73% in plaintext; 73% with a 64-bit ring). The AdamW model is much less sensitive to it.

Setting	images	correct
Plaintext, float (PyTorch)	100 / 192	76 / 138 (71.9%)
Trio 3PC, standard configuration	100	71
Trio 3PC, reduced-slack truncation (TRUNC_APPROACH=1)	100	70
2PC ABY2, single batch, all 30 variants	100	62–72
2PC ABY2, multi-batch, all 30 variants	192	118–131
2PC ABY2, A_UNKNOWN=0, single / multi-batch	100 / 192	64–72 / 116–133
2PC ABY2, public weights, single / multi-batch	100 / 192	63–73 / 115–131
2PC ABY2, CUT on / off (plain family, 12 runs each)	1,752	1,134 / 1,138

Table 14: Final accuracy with the AdamW-trained ResNet50 on the first CIFAR-10 test images.

9 The all_opt configurations on ImageNet

The artifact’s single-batch 2PC configurations (`measurements/configs/artifacts/triad/2pc/single_batch`) combine the online optimizations of the protocol: six configurations, A2bits and reshared, each in the three use cases of Section 2: UC1, weights known to none (A_UNKNOWN=0); UC2, weights known to the model owner (A_UNKNOWN=1, here also known in preprocessing, MODELWEIGHTS_KNOWN_DURING_PREPROCESSING=1); and UC3, weights known to all (public weights). Each runs ResNet50 on one ImageNet image with the three adders (FUNCTION_IDENTIFIER=87, 187, 287) and with COMPRESS=0, 1: 36 builds. As in the configurations, the model and the image are dummies (MODELOWNER=-1, DATAOWNER=-1); the runtime does not depend on their values.

Builds. *As given:* the configuration unchanged (hpmc 161cf1d); its conv triples use CHEETAH’s HomConv2DSS. *Optimized:* the configuration plus every optimization of this report (final code: hpmc 92a2c0e, ConvTriple 8d8f244, flexNN 417f12b; rounds 1–3 at hpmc 1421df0): CHEETAH_CONV_PACKED=1 with the pipeline, CUT_FRACTIONAL_BITS_OPT=1, ADDITIONAL_RELU_THREADS=24 (equal to the configuration’s ADDITIONAL_GEMM_THREADS, as the two share one pool), 100,000-word buffers, and A2B_CONV_BAKE=1 for the A2bits configurations; everything else of Sections 3 to 5 and of the rounds below is on by default (the communication-optimized output repacking of round 4 is a separate variant); Table 15 lists every switch. RNG_AHEAD=1, part of the optimized set until this last round, is left out: at the final code it saves the Zen 4 hosts about 0.04s of online time (median over the 36 builds) but costs the Zen 3 hosts up to 0.17s (UC3, COMPRESS=1: 0.51 against 0.36s); a profile puts that in the online draws, most likely waiting for the ring’s producer thread. Each build runs three times per machine pair (median): the as-given builds alternating with the final code, the optimized builds in a separate series.

Correctness. The same flags on CIFAR-10 with the AdamW model and real data (P0 / P1 as owners, 10 images) classify 3–7 images correctly for every configuration with COMPRESS=0 at the final code (plaintext: 6), including all UC2 builds. The outputs change only where a change alters the randomness that the probabilistic truncation sees (the bake’s privacy fixes of rounds 4 and 7, P1’s prescribed shares in UC2 A2bits in round 6); every other change leaves all 18 output hashes as they were. COMPRESS=1 evaluates the ReLU on bits 12–19 only (REDUCED_BITLENGTH_m=12, k=20). With FRACTIONAL=5 the carry dropped from the 12 low bits flips the sign of activations below 128.0, and even the plain RCA build without any of our changes classifies 0 of 10. It is a runtime setting only, and we keep it running rather than accurate. Its A2bits builds aborted with the element-parallel levels: the A2B bake committed conv masks for 8 of the 32 slices, so most outputs drew fresh masks. The bake now requires a full-width ReLU, and COMPRESS=1 runs the A2B unbaked.

The A2bits configurations *as given* are not correct. They set A2B_ONLINE_OPT=1, which precomputes the A2B’s secret operand $[c] = \text{bool}(-\lambda)$ by a Boolean addition in preprocessing, but not the bake (A2B_CONV_BAKE), and without it the preprocessing pass generates the MSB adder’s Beaver triples for the Boolean addition’s input while the online phase uses its output $[c]$. On CIFAR-10 with the AdamW model, the as-given A2bits flags classify 0–2 of 10 images in all three use cases and every adder, as do the optimized flags without the bake (same output hashes as as-given in UC3); with the bake they classify 3–6. The reshared configurations do not use A2B_ONLINE_OPT and classify 4–7 of 10 as given.

Why the bake remarks ReLU inputs. A ReLU on $m = v + \lambda$ runs its MSB adder on $s_1 = \text{bool}(m)$, public, and $s_2 = \text{bool}(-\lambda)$, secret, since $\text{bool}(m) \boxplus \text{bool}(-\lambda) = \text{bool}(v)$. A2B_ONLINE_OPT computes $[c] = \text{bool}(-\lambda)$ in preprocessing, from the parties’ $\text{bool}(-\lambda_0)$ and $\text{bool}(-\lambda_1)$ by a Boolean addition (31 levels of AND gates). Two facts fix *when* this can happen. The adder’s Beaver triples are defined by the masks of its input wires, $[c]$ among them, and are requested while the preprocessing pass walks the circuit. And that pass is a single non-interactive sweep that fixes masks, sends one-way messages and records triple requests; the triples are

generated in bulk afterwards. So $[c]$ cannot be computed inside the pass, which would need 31 interactive rounds mid-sweep. It cannot be computed after the pass either, which is the as-given configuration: its triples then belong to a different mask. It must exist *before* the pass, and with it every ReLU input's mask λ .

The bake therefore commits the masks first. Each party i fixes a mask ℓ_i per ReLU input before the pass, the Boolean addition runs on $\text{bool}(-\ell_i)$, and the pass then has to deliver every ReLU input under the mask $\ell = \ell_0 + \ell_1$. How depends on the layer that produces the input:

- **A convolution or FC layer with secret weights** (UC1, UC2) ends with a message that re-masks its output under a mask each party draws: the party draws ℓ_i instead. With weights known in preprocessing, P1's triple share is chosen so that its truncated output mask is ℓ_1 ; such masks lie in the SecureML truncation's image (top FRACTIONAL bits zero), so P1 commits ℓ_1 there, uniformly. In the CHEETAH ResNet50 layout 8.00 of the 9.01 million ReLU inputs come straight from a convolution (with BatchNorm fused into it).
- **A residual sum** (0.80 million inputs): the addend computed last draws $\ell_i - \lambda'_i$, where λ' is the other addend's mask, so that the sum carries ℓ (round 5). P1's masks in UC2 must lie in the image, which $\ell_1 - \lambda'_1$ does not for a free λ'_1 ; so P1 commits the other addend's mask m_b as well (a convolution draws it as a PRF value in the image, a ReLU's outputs are committed anyway) and commits $\ell_1 = m_a + m_b$ for the sum, exactly the mask the local addition gives; the partner then draws m_a (round 6).
- **A convolution with public weights** (UC3) is local, so its output mask is $W * \lambda_{\text{in}}$, a linear function of the previous bit injections' masks; choosing those so that $W * \lambda_{\text{in}} = \ell$ is a linear system over $\mathbb{Z}_{2^{32}}$ without a solution in general. But every ReLU input's mask is a function of the input's masks and the earlier bit-injection masks alone, so the pass first runs the network over the masks, with each ReLU recording its input masks and outputting committed bit-injection masks, then runs the Boolean addition on the actual masks, and then the real sweep (round 4). With secret weights the same mask-only forward lets the conv triples start before the pass (round 5).
- **Every other producer** fixes λ without such a choice: in UC1 and UC2 the stem's pooling, whose BatchNorm passes its output on (0.20 million inputs). Such an input is moved onto the committed mask: party i sends $\delta_i = \ell_i - \lambda_i$ in preprocessing, and online both parties add $\delta_0 + \delta_1$ to m , which gives $v + \ell$. Since ℓ_i is uniform and known only to party i , δ_i is a one-time pad of λ_i ; the price is one ring element per value and party, in preprocessing only: 1.5 MiB.

Each committed mask must mask exactly one value, or the public masked values and remask messages reveal differences of secret values or masks; and a mask may lie in the truncation's image only where nothing the other party sees is masked by it alone (UC2, where P0 forms the masked value from its own share). Rounds 4, 6 and 7 below fixed four violations of the first version. In preprocessing every ReLU input that its producer moved is checked against its committed mask. In the first version, UC1 and UC2 remasked 1.00 million inputs (7.7 MiB) and UC3 all 9.01 million ($9.01 \cdot 10^6 \cdot 2 \cdot 4$ bytes = 69 MiB, exactly what the round-3 builds sent more than the as-given ones); now UC1 and UC2 remask 0.20 million and UC3 none.

What the bake costs. The reference is the same build with A2B_CONV_BAKE=0: the as-given A2B, which runs the same Boolean addition (after the pass) and computes wrong signs, but sends what an A2B without the bake sends. At the final code on flare / polynize (three interleaved runs per build, all nine A2bits builds with COMPRESS=0; with COMPRESS=1 the bake is off):

- **Communication.** The triple generation and the online phase send the same bytes with and without the bake, in every build. The preprocessing pass sends 0.80 MB more in each direction in UC1 and UC2, the remask of the stem's 200,704 ReLU inputs (1.53 MiB, 0.1% of their 1.0–1.8 GiB of preprocessing), and nothing more in UC3. So the bake is free in communication with public weights, and costs the stem's remask with secret weights; free there too would need the stem's pooling truncation (or its BatchNorm) to commit its output mask (Section 10).
- **Preprocessing time.** With secret weights -0.10 to $+0.09$ s (UC1 0.05–0.10 s less, UC2 0.03–0.09 s more): the bake moves the Boolean addition from after the pass into the OT phase, next to the conv triples, and adds the commitment (and in UC2 P1's residual commitment). With public weights 0.06–0.16 s more: the mask-only forward, about 0.3 s of serial work, partly hidden.
- **Online time.** 0.00–0.07 s less: the baked A2B reads the committed $[c]$ directly, where the unbaked one retrieves the Boolean addition's outputs. A first measurement (round 6) had put the bake at up to 0.1 s more online and 0.2 s more preprocessing: the baked A2B prepared its values serially, and the commitment was derived on one thread; round 7 removed both.

Where the time went. In a profile at `hpmc aeab3ae` (before the last two changes below), preprocessing was bound by OT generation: in a CPU profile over all threads, `ferret`’s LPN step took 31–44%, the MITCCRH key schedules 8–11% and MPCOT 10–13%. Per A2bits RCA inference the OT packs’ setup cost 0.9s, the bake’s Boolean addition 1.5s (31 carry levels over 9.0 million values; one level pays a `ferret` extension of all 32 packs), the conv triples 0.7s and the network’s preprocessing pass 0.5s; PPA4 added 3.1s of Beaver tuples. 84% of the online CPU samples were idle workers: RCA’s 1,714 rounds at about 0.3ms each are close to its floor, and disabling the NIC’s interrupt coalescing, which cut the ping from 0.25 to 0.08ms, shortened the RCA online phase by only 7%. Table 17 shows the final code.

Changes.

- **Worker wake-ups.** Between two circuit levels a worker spun about 100µs and then slept on a `futex`, shorter than a round trip, so every level woke 24 threads. Workers now wait about 20ms, on the counter’s own cache line: with the counter next to the completion count, every completion invalidated the line for all spinning workers, which slowed the Zen 3 hosts’ UC3 builds by up to 35%. On AMD CPUs the wait uses `MONITORX/MWAITX`, which parks the thread on the cache line without taking issue slots from its SMT sibling.
- **Vectorized GEMM accumulation** (`A_KNOWN=1`). The online GEMM called a share-level kernel per multiply-accumulate on $\{m, \lambda\}$ structs, about 0.2 MAC per cycle. The kernel of each variant is a sum of plain ring products, so it now runs as $C = XY^T$ on `uint32` matrices with a register-blocked 4×32 kernel (AVX-512 or AVX2), with the weight operand cached across a layer’s calls. The results are bit-identical; with weights known in preprocessing, P1’s accumulation, which its mask/send never reads, is skipped. The conv layers’ online time falls by a quarter on Zen 3; on Zen 4 the accumulation was not on the critical path.
- **The A2B bake** is restricted to full-width ReLUs (above).
- **Conv triples alongside the preprocessing pass** (`CHEETAH_CONV_ASYNC`). The conv triples used to start after the pass, whose single thread fixes their input masks. Now the batched HE pipeline starts on its own thread together with the pass and takes each layer as soon as the pass has recorded its masks; the rest of the preprocessing waits for it before the next generator uses the channels. Only the layers the pass reaches last leave a tail.
- **Parallel preprocessing levels** (`STREAM_PARALLEL_PRE`). The A2B, adder and bit-injection levels of the pass run on the worker pool like online. The pass’s append-only streams (triple types and triple inputs, the pre-send buffer, stored outputs) get per-worker cursors, reads use the online cursors, and every worker’s use of each stream is checked against the first element’s. The GEMM levels stay serial. Both changes leave all 18 CIFAR-10 output hashes and two multi-batch hashes unchanged.

More HE/OT threads help the OT-heavy builds: `CHEETAH_THREADS=64` instead of the configurations’ 32 (64 OT packs) shortens preprocessing by 8% (median) on Zen 4 and 4% on Zen 3, by up to 20% for PPA4, but lengthens it by up to 30% for the UC3 RCA builds, where setting up 64 packs costs more than it saves. Two `ferret` threads per pack, busy-polling the receive path and 31 instead of 24 workers did not help.

Results. Figure 16 shows all 36 builds; Table 16 summarizes them. At the final code every build is faster than as given online, on both pairs, and in preprocessing all but two UC3 builds with `COMPRESS=1` on Zen 3, which take 1–2% longer (1.88 against 1.86s and 2.01 against 1.97s): rounds 4–7 did not change their code, and the difference is within the run-to-run spread of these 2-second runs. In UC1 and UC2 the packed, pipelined and concurrent conv triples with the exact wire format (since round 5 alongside the OT phase) cut preprocessing from 11.7–17.3s to 1.7–5.5s on the Zen 4 pair ($2.9\text{--}7.1\times$) and from 15.1–21.9s to 2.7–6.9s on the Zen 3 pair ($3.0\text{--}6.0\times$). The online phase falls from 0.77–1.55s to 0.34–0.63s ($2.0\text{--}3.0\times$; Zen 3: 0.72–1.73 to 0.24–0.63s, $2.2\text{--}4.2\times$). In UC2, with the weights known in preprocessing, the online phase takes 0.34–0.56s on Zen 4 and 0.24–0.48s on Zen 3.

UC3 needs no linear-layer triples, so its preprocessing is OT-bound and changes least: $1.1\text{--}1.2\times$ faster ($0.98\text{--}1.1\times$ on Zen 3), with the online phase $1.5\text{--}2.2\times$ faster ($1.4\text{--}2.2\times$ on Zen 3). Before the parallel preprocessing levels, the Zen 3 hosts’ A2bits builds with `COMPRESS=1` preprocessed 0.11–0.14s *slower* than as given (five interleaved runs each). A one-flag-off ablation placed all of it in `ADDITIONAL_RELU_THREADS`, whose per-call stream-cursor checks slowed P0’s last preprocessing step from 0.32–0.34 to 0.44–0.49s. Unchecked versions for the preprocessing share removed that but slowed the `COMPRESS=0` builds by as much, a code-layout effect we did not keep. The parallel preprocessing levels now more than make up for it. As-given A2bits also skips the A2B bake and is not correct (Correctness).

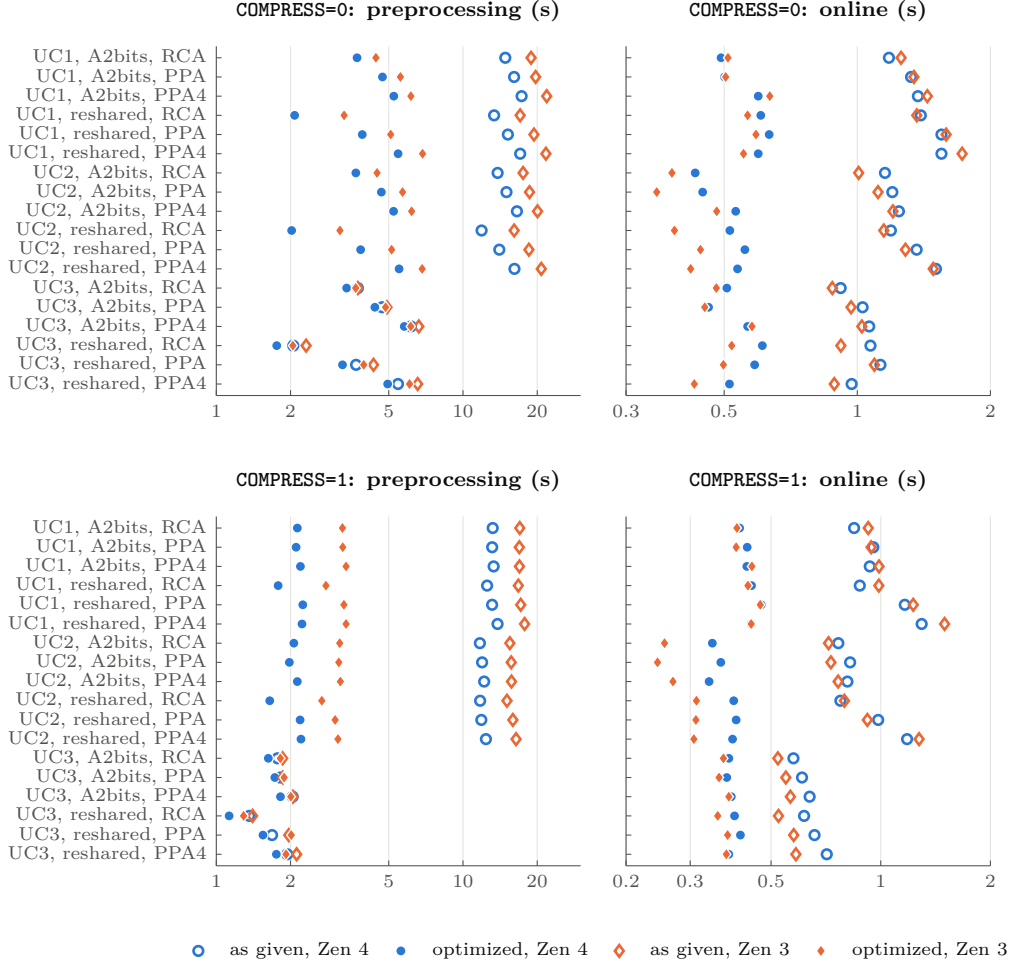


Figure 16: The 36 all_opt builds on ImageNet (dummy weights) by use case, as given and optimized, on both machine pairs (log scale; median of three runs).

Communication. Figure 17 shows the traffic of the same builds as P0 counts it; P1’s counters mirror P0’s, and the reruns and the Zen 3 pair send the same bytes. In UC1 and UC2 the optimized builds send $1.35\text{--}1.86\times$ less in preprocessing with **COMPRESS=0** (1,323–3,225 to 790–2,150 MiB) and $1.61\text{--}1.92\times$ less with **COMPRESS=1** (1,219–2,353 to 696–1,308 MiB). The conv triples account for nearly all of it: in UC2 they fall from about 1,000 to 479 MiB (Table 3), in UC1, whose AB triples need both cross products, from 2,003 to 958 MiB. With the weights known in preprocessing, both UC2 builds also send P1’s share corrections, 4 bytes per conv output (42 MiB). The OT-based tuples are unchanged, except that with **CUT_FRACTIONAL_BITS_OPT** PPA4 needs fewer multi-input AND tuples: 12 MiB less for resharded and 74 MiB less for A2bits, where the bake lets the cut skip the b -product gates. The network’s preprocessing pass changes by -67 to $+2$ MiB in UC1 and UC2 (of the bake’s remark only the stem’s ReLU inputs are left, 1.5 MiB: the residual sum’s 6.1 MiB are baked since rounds 5 and 6). UC3 needs no linear-layer triples, and its traffic is unchanged. In round 3 its A2bits builds with **COMPRESS=0** sent 69 MiB more than as given: no ReLU input comes straight from a masked conv layer, so the bake rebased every one of the 9.0 million inputs onto its committed mask, which the as-given builds skip (and compute wrong signs, see Correctness). The mask-only forward of round 4 bakes them without that message, so the final builds send exactly what the as-given ones do, and compute correctly. Online, **CUT_FRACTIONAL_BITS_OPT** removes 4–8% of the traffic of the UC1 and UC2 builds with **COMPRESS=0** (A2bits PPA4: none). The cut needs **TRUNC_DELAYED=0**, so it does not apply to UC3; with **COMPRESS=1** the online traffic does not change.

Round 4. Four further changes; the measurements in rounds 4–6 are from flare / polynize, the rerun of all 36 builds at the final code (Tables 16 and 17, Figs. 16 and 17) from both pairs.

1. **P1’s bake in the pass.** The conv triples of resharded PPA4 did not run at half speed next to the pass: they waited for P1, whose pass took 1.28–1.43s against P0’s 0.38–0.50s. P1 bakes the reshare bits into every conv output mask in the (serial) GEMM level, and with **CUT_FRACTIONAL_BITS_OPT** the PPA4

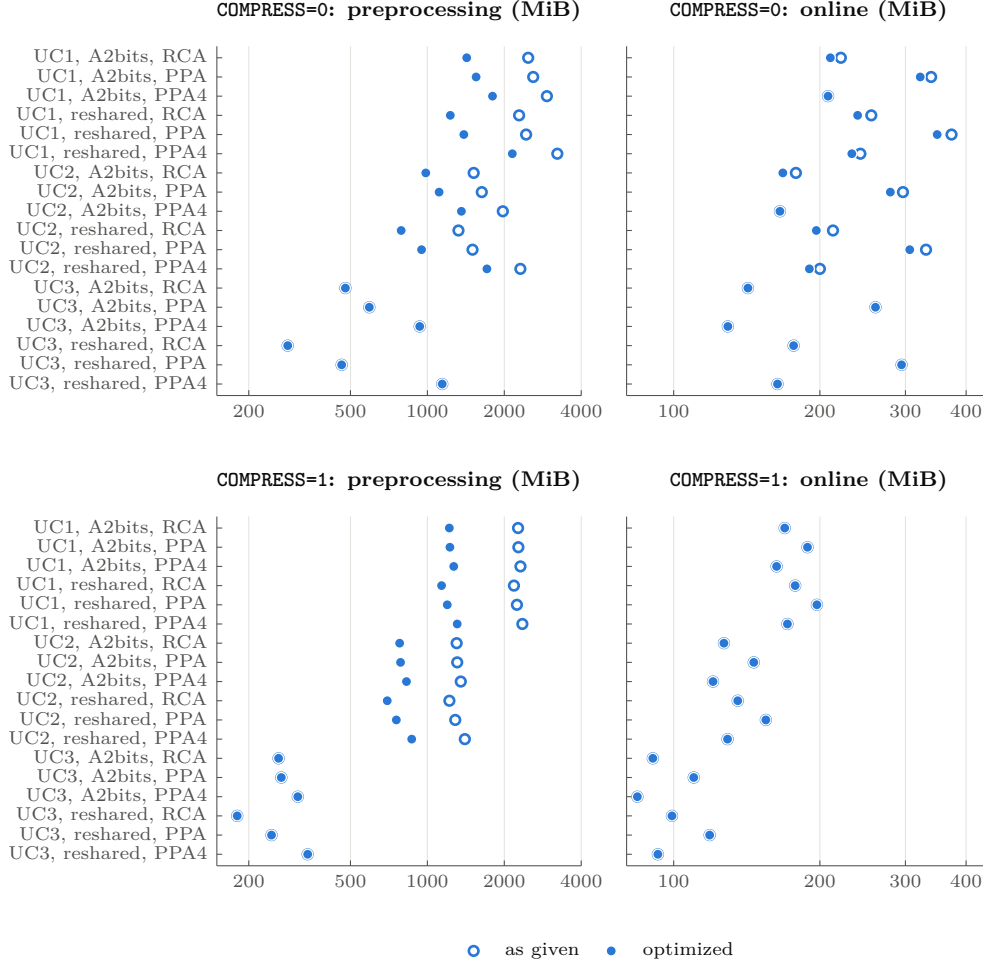


Figure 17: Traffic of the 36 all_opt builds on ImageNet (dummy weights) by use case, as given and optimized: MiB sent plus received by P0, i.e. in both directions (log scale). Preprocessing counts the triple generation (HE, OT, key exchange) and the network’s preprocessing pass. Every run and both machine pairs send the same bytes.

slice ranks were recomputed for each of the 32 slices of each of the 7.3 million outputs. With the ranks fixed at compile time P1’s pass takes 0.55–0.66 s, and reshared PPA4 preprocesses in 5.5–5.7 s instead of 6.0–6.4 s (UC1 and UC2), with the same outputs bit for bit.

2. **Privacy of the bake.** The committed masks are addressed by the next A2B’s position, and every convolution took them. So the downsample convolution and the one after it masked their outputs with the same masks (1.5 million values: the differences of secret activations were public), and at the residual ReLU the remask message $\ell_i - \lambda_i$ gave away the downsample convolution’s mask, since ℓ_i had also masked its partner. And the committed masks replayed the generator stream the passes then drew their own masks from. Now only a convolution feeding a baked ReLU takes committed masks, the others draw fresh ones, and the committed values come from a separate stream. Neither change costs traffic or time; the results of rounds 1–3 stand.
3. **UC3 bake for free.** The mask-only forward described above: UC3 A2bits sends 69 MiB less in preprocessing (546 to 478 MiB for RCA, 1,002 to 933 for PPA4, –7 to –13%) in the same time (3.52–5.82 against 3.54–5.92 s), and its online phase gets slightly faster (0.56–0.64 against 0.63–0.68 s), since no ReLU input is moved. In preprocessing every ReLU input’s mask is checked against the mask-only forward’s; none of the 49 ReLUs differs.
4. **Output repacking,** the communication-optimized variant (CHEETAH_CONV_REPACK=1, Section 3.1): 285 instead of 479 MiB per HE product, preprocessing traffic –18 to –31% in UC1 (1,229–2,150 to 845–1,766 MiB) and –11 to –24% in UC2 (790–1,710 to 598–1,518 MiB), for 1.3–1.7 s more preprocessing; the online phase is unchanged.

Round 5. Two more changes.

1. **Conv triples alongside the OT phase.** During the OT phase only about 20 of the 64 hardware threads are busy (sampled every 50 ms), while the conv triples ran with the pass and after it. Every ReLU's outputs now take committed masks (values of a PRF under the party's key, by position), so every convolution's input masks are known before the pass: the pass first runs the network over the masks, where each convolution records its triple's inputs and stops, then starts the conv triples on four channels of their own, runs the OT phase (moved into the pass), and then the real sweep, which checks every convolution's inputs against the recorded ones. With all 32 threads the conv triples slowed the OT phase down by about as much as they gained, with 16 throughout they became the tail of the Zen 3 builds with short OT phases (up to 0.7 s slower); they now take 16 while the OT phase runs and all 32 after it. Against round 4 on Zen 4 preprocessing takes 0.03–0.34 s less over the 12 UC1 and UC2 builds; over all 36 builds, the round-5 code preprocesses a median 0.15 s faster than round 3 on Zen 4 (up to 0.8 s) and 0.03 s on Zen 3, where a few builds with short OT phases are up to 0.11 s slower. The online phase gets 0.02–0.07 s faster, since the bit injections no longer draw masks.
2. **Residual sums baked.** The layer computed last of a residual sum's two addends draws $\ell - \lambda'$, where λ' is the other addend's mask, so that the sum carries ℓ . In UC1 this removes the residual sum's remask (6.1 MiB); in UC2 P1's masks must lie in the SecureML truncation's image, so P1 still sent its half alone (3.1 MiB saved; round 6 removes the other half). The outputs are unchanged bit for bit. In preprocessing, every baked ReLU input's mask is checked against its committed mask; the check showed that with dummy weights P1 holds a bias mask that the UC2 bake cannot compensate, so P1's A2B inputs were off after biased convolutions in the dummy-weight ImageNet runs (the timing measurements are unaffected).

Round 6. In UC2, P1 now also bakes the residual sum. P1's partner cannot draw $\ell_1 - \lambda'_1$ for a free λ'_1 , since with weights known in preprocessing P1's conv masks are the truncation's image of its prescribed triple share. So the other addend's P1 mask is committed as well: a convolution producing it draws a PRF value in the image (uniform there, like a fresh mask, so the prescribed share stays uniform), a ReLU producing it has committed bit-injection masks anyway (round 5). P1's committed mask of the sum is then $\ell_1 = m_a + m_b$, exactly the mask the local addition gives, and the partner draws m_a , again in the image. Which sums are committed follows from the network alone, so both parties agree on who sends what. This removes the remaining 3.1 MiB (ImageNet, every UC2 A2bits build with COMPRESS=0: 3.21 MB fewer received by P0, nothing else changes); on the CIFAR ResNet50, whose every block has a residual sum (4 produced by a convolution, 12 by a ReLU), the UC2 pass traffic falls from 16.8 to 12.3 MB, and P1's check of every baked ReLU input passes with the real weights. Convolutions now cost the A2B no message at all: in UC1 and UC2 only the stem's pooling output is remasked.

The round also fixes a leak in UC1: the committed masks were narrowed to the truncation's image in every build with TRUNC_DELAYED=0, but in UC1 P1's masks are free draws that hide the truncated share it sends, so a narrowed mask revealed part of it (about one bit per value on average); only builds with weights known in preprocessing narrow now. UC1's outputs do not depend on these masks and stay bit for bit the same. Neither change costs time: against round 5, the 36 builds preprocess a median 0.01 s slower on Zen 4 (−0.21 to +0.10 s) and as fast on Zen 3 (−0.12 to +0.16 s), and the online phase stays within 0.04 s. (An A/B series right after the change had put UC1's A2bits RCA build 0.1–0.2 s behind round 5, all of it in P1's OT setup; in the final series the identical binary ran 0.05–0.1 s *faster* than round 5, so that was series-to-series variation, not the change.)

Round 7. The bake's runtime and one more privacy fix.

1. **Repeated committed masks.** The counter-mode PRF behind the committed masks (the mask-only forward's truncation masks since round 4, every committed ReLU output mask since round 5, UC2's residual masks of round 6) took its block index and slot as $k/\text{BUFFER_SIZE}$ and $k \bmod \text{BUFFER_SIZE}$ of a macro without parentheses: $k/512/32$ and $(k \bmod 512)/32$. Each value came back for 32 consecutive masks, 16 distinct ones per 16,384, so each committed mask hid many values and the other party saw differences of secret activations. Masks never change the work, so no timing, traffic or accuracy number was affected; it showed when the commitment below drew from the same PRF and P1's narrowed UC2 masks came out tiny (SecureML truncation wraps, 0–2 of 10). The macro is parenthesized, and the PRF checks its outputs before the committed masks are drawn. The CIFAR outputs of all builds with committed masks change; all 18 classify 3–7 of 10.
2. **Baked A2B on the pool.** The baked A2B read $[c]$ through one global cursor, so its prepare and complete levels ran serially, where the unbaked A2B runs them on the worker pool. Each value now reads its slices by index.
3. **Parallel commitment.** The committed masks come from the PRF by slot, on 16 threads, in the OT phase.

Against round 6, preprocessing is unchanged (median ± 0.00 s on Zen 4, -0.01 s on Zen 3) and the online phase a median 0.01 s and up to 0.10 s faster.

What is left. Table 17 splits the time of six builds at the final code; the ranges below are over all 18 builds with COMPRESS=0. With secret weights, preprocessing is now the OT phase (1.4–4.7 s, inside the pass), the pass’s own work (0.33–0.58 s, including its mask-only forward), the steps after it (0.22–0.56 s) and 0.1–0.3 s of key exchange, input preparation and gaps; the conv triples run alongside the OT phase and finish before the pass does.

1. **OT generation** remains the largest cost. Every build pays 0.84–1.04 s to set up the 32 ferret packs (about 0.1 s more with secret weights, whose conv triples run alongside); the tuples after that take 0.26–0.34 s for RCA but 1.6–3.6 s for PPA4, whose Beaver 3- and 4-tuples need up to six more extension rounds of all 32 packs. In the round-3 CPU profile, ferret’s LPN step took 33–41% of all samples, the AES key schedules of the correlation-robust hash 8–11%. The last extension round is only partly used, but that wastes at most one round (about 0.3 s for PPA4). A fixed-key hash would remove most of the key schedules, but that is a security decision (Section 10).
2. **The bake’s Boolean addition** costs the A2bits builds 1.41–1.53 s: 31 dependent COT-multiplication rounds of 15–22 ms plus an extension round. A log-depth adder would need 5–6 rounds but more AND gates, and so more OTs.
3. **The conv triples** no longer show: in round 3 the HE ran 0.28–0.60 s past the pass (reshared PPA4: 1.0–1.1 s, from P1’s serial bake, fixed in round 4). Running alongside the OT phase, they compete with it for cores and memory bandwidth, which the longer OT setup shows.
4. **Serial work** in the pass: the GEMM levels (the weights-known-in-preprocessing path pushes into vectors there) and the bake’s input preparation still run on one thread; with public weights the mask-only forward adds 0.3 s to the A2bits pass (0.55–0.74 against 0.24–0.30 s).
5. **Online**, RCA’s 1,714–2,012 rounds cost 0.16–0.19 s of waiting. Of the rest, the main thread spent 0.11 s handing levels to the pool, 0.05 s transposing bits for the A2B serially, and 0.06 s in layer glue (copies, residual additions, freeing buffers) in the round-3 profile.
6. **Communication:** output repacking (round 4) removes 40% of the conv-triple traffic at 1.3–1.7 s of CPU; it pays off below about 1.2 Gbit/s (UC2) or 2.1 Gbit/s (UC1). Of the bake’s remask, only the stem’s pooling output is left (1.5 MiB, Section 10).

flag (optimized setting)	effect	where
<i>set in the optimized builds</i>		
CHEETAH_CONV_PACKED=1	conv triples with the packed HE evaluator (several images and output channels per ciphertext, lazy transforms, exact wire format, concurrent chunks) instead of CHEETAH's HomConv2DSS	Section 3
CHEETAH_CONV_PIPELINE=1	all convolutions in one pipelined call: encryption, evaluation and decryption of consecutive layers overlap	Section 3
CUT_FRACTIONAL_BITS_OPT=1	the MSB adder skips the bits the preceding SecureML truncation fixes (TRUNC_DELAYED=0): fewer AND tuples for PPA4, 4–8% less online traffic	Section 5
ADDITIONAL_RELU_THREADS=24	the elements of each ReLU level (MSB adder, A2B, bit injection) run on the worker pool of the GEMM threads	Section 5
SEND_BUFFER, RECV_BUFFER = 100,000	network buffers of 100,000 instead of 10,000 words	Section 5
A2B_CONV_BAKE=1 (A2bits)	every ReLU input's mask is committed before the preprocessing pass, which A2B_ONLINE_OPT needs to be correct	below
<i>new defaults since the configurations (on)</i>		
GEMM_POOL_SPIN = 2 ²⁰ , GEMM_POOL_MWAITX=1	pool workers wait across a round trip on their own cache line, with MONITORX/MWAITX on AMD	Changes
GEMM_FAST=1	online GEMM of A_KNOWN=1 as a register-blocked uint32 matrix product	Changes
CHEETAH_CONV_ASYNC=1	conv triples alongside the preprocessing pass, each layer once its masks are known	Changes
STREAM_PARALLEL_PRE=1	the preprocessing pass's ReLU levels on the worker pool	Changes
CHEETAH_CONV_EARLY=1	a mask-only forward records every conv's triple inputs; the conv triples run alongside the OT phase (CHEETAH_CONV_EARLY_THREADS: 16 of 32 threads during it, all after it)	round 5
A2B_BAKE_MASK_PASS=1	UC3: the Boolean addition runs on the masks a mask-only forward computes, so no ReLU input is remasked	round 4
A2B_BAKE_RESIDUAL=1	residual sums carry the committed mask (in UC2, P1 also commits the other addend's mask)	rounds 5, 6
<i>variants, off</i>		
CHEETAH_CONV_REPACK=1	communication-optimized conv triples (output repacking, $N = 8192$): −40% conv traffic, +1.3–1.7s	round 4
A2B_BAKE_BN=1	a BatchNorm before a ReLU takes the committed masks (unfused BatchNorm only)	round 5
RNG_AHEAD=1	the party's own random stream produced ahead by a thread: −0.04s online on Zen 4, up to +0.17s on Zen 3	Builds
CHEETAH_THREADS=64	64 OT packs: −8% (median) preprocessing on Zen 4, up to +30% for UC3 RCA	Changes
TRIPLE_GPU (ConvTriple)	the packed pipeline's evaluator on a GPU, bit-identical: conv triples 2.1–2.3× faster	Section 3.3
GEMM_FAST_GPU=1	GEMM_FAST's online product on the GPU, exact; no gain at batch size one	Section 3.3

Table 15: The optimized ImageNet builds: flags set on top of the all_opt configurations, the defaults this work added after the configurations (hpmc 161cf1d), and the variants left off. Everything else, e.g. the OT changes of Section 4 and TCP_NODELAY, was already in the code the configurations were given with.

builds	pair	preprocessing (s)		online (s)	
		as given	optimized	as given	optimized
UC1, known to none, COMPRESS=0 (6)	Zen 4	13.4–17.3	2.1–5.5	1.18–1.55	0.49–0.63
	Zen 3	17.1–21.9	3.3–6.9	1.26–1.73	0.50–0.63
UC1, known to none, COMPRESS=1 (6)	Zen 4	12.5–13.8	1.8–2.2	0.84–1.29	0.41–0.47
	Zen 3	16.8–17.8	2.8–3.4	0.92–1.50	0.40–0.47
UC2, known to one, COMPRESS=0 (6)	Zen 4	11.9–16.5	2.0–5.5	1.16–1.51	0.43–0.56
	Zen 3	16.1–20.8	3.2–6.8	1.01–1.49	0.35–0.48
UC2, known to one, COMPRESS=1 (6)	Zen 4	11.7–12.4	1.6–2.2	0.77–1.18	0.34–0.40
	Zen 3	15.1–16.4	2.7–3.2	0.72–1.27	0.24–0.31
UC3, known to all, COMPRESS=0 (6)	Zen 4	2.1–6.2	1.8–5.8	0.92–1.13	0.46–0.61
	Zen 3	2.3–6.6	2.0–6.1	0.88–1.09	0.43–0.58
UC3, known to all, COMPRESS=1 (6)	Zen 4	1.4–2.0	1.1–1.8	0.58–0.71	0.38–0.41
	Zen 3	1.4–2.1	1.3–2.0	0.52–0.58	0.36–0.38

Table 16: The all_opt builds on ImageNet by use case (A2bits and reshared, three adders each), as given and optimized at the final code (round 7; median of three runs). Per-build values: docs/TRIAD_ALL_OPT.md.

build	preprocessing (s)								online		
	total	OT setup	tuples	bake add.	pass	conv tail	after	other	(s)	rounds	wait (s)
UC1, A2bits, RCA	3.62	0.99	0.26	1.47	0.37	0.00	0.22	0.31	0.50	1,818	0.16
UC1, reshared, PPA4	5.48	0.99	3.60	0.00	0.38	0.00	0.40	0.11	0.60	779	0.14
UC2, A2bits, RCA	3.72	0.99	0.26	1.53	0.33	0.00	0.28	0.33	0.48	1,714	0.19
UC2, reshared, PPA4	5.47	1.00	3.55	0.00	0.39	0.00	0.44	0.10	0.53	675	0.14
UC3, A2bits, RCA	3.58	0.95	0.29	1.53	0.55	0.00	0.16	0.10	0.53	1,963	0.17
UC3, reshared, RCA	1.67	0.86	0.34	0.00	0.24	0.00	0.16	0.07	0.60	2,012	0.17

Table 17: Where the time of six builds goes at the final code (round 7, COMPRESS=0, ImageNet, flare / polynize, phase timestamps, median of two runs). Tuples: OT-based triples and tuples after the OT setup; bake add.: the A2B bake’s Boolean addition; pass: the network’s preprocessing pass including its mask-only forward, without the OT phase that runs inside it (secret weights) or the Boolean addition (UC3 A2bits); conv tail: the part of the conv triples still running after the pass (since round 5 they run alongside the OT phase, and none is left); after: the rest of the preprocessing (COT, multiplexer, FC triples, triple bookkeeping); other: key exchange, the bake’s input preparation, starting the conv triples, and gaps. Wait: online time spent waiting for the other party.

10 Limitations and open problems

- **Truncation.** ABY2 has no reduced-slack or exact truncation yet. It is not needed for accuracy with the AdamW model, but it would remove the truncation noise (Section 8).
- **Weights known in preprocessing.** The multi-batch a2b bake still excludes `MODELWEIGHTS_KNOWN_DURING_PREPROCESSING`, whose prescribed shares assume one word per register.
- **Ciphertext occupancy.** At 7×7 feature maps, inputs are 5–19% data and outputs 6–24%. Packing the sparse outputs (automorphisms, $N = 8192$ with a special prime) is implemented as the communication-optimized variant `CHEETAH_CONV_REPACK` (round 4 in Section 9): 285 instead of 479 MiB per HE product for 1.3–1.7s more preprocessing, worth it below about 1–2 Gbit/s. Packing the inputs remains open.
- **A2B bake.** UC1 and UC2 still remask the 0.20 million ReLU inputs after the stem’s pooling (1.5 MiB in preprocessing). Committing the pooling’s truncation mask, or a BatchNorm’s output mask like a convolution’s, would remove that; a BatchNorm bake works mechanically but changed the outputs of an unfused multi-batch check for reasons not yet understood, so it is off.
- **OT extension.** On the CPU, ferret’s LPN step and the AES key schedules of the correlation-robust hash are the largest costs of the Boolean preprocessing; a fixed-key hash is a security decision. On a GPU both run there (Section 3.3); what remains on the CPU is the packs’ setup (base OTs, IKNP, first extension), the latency-bound Boolean addition of the A2bits builds, and the few consumers that need the COT blocks themselves. The online phase measured 0.02–0.07s higher with the GPU OT path on the shared workstation, for reasons not yet found; a two-machine GPU setup would settle it.

11 Conclusion

The largest runtime gains came from doing less redundant work in the HE evaluator (packing, lazy transforms, pipelining). The largest communication gains came from sending only information-bearing bits: the exact wire format, full BatchNorm ciphertexts, and OTs whose message bits carry several products. In the online phase, latency rather than bandwidth dominates, so the gains came from Nagle-free sockets, fewer rounds and parallelizing the per-round work at batch size one. Seeded, bit-reproducible runs made every change measurable and exposed the bugs in Section 7. Moving the A2B to preprocessing (Section 9) needs every ReLU input’s mask fixed before the preprocessing pass; most producers can commit their masks for free, and the care it takes is privacy bookkeeping: each committed mask may mask one value only, and a mask narrowed for a truncation only where the other party never sees what it hides.

References

- [1] A. Patra, T. Schneider, A. Suresh, H. Yalame. ABY2.0: Improved mixed-protocol secure two-party computation. USENIX Security 2021.
- [2] Z. Huang, W. Lu, C. Hong, J. Ding. Cheetah: Lean and fast secure two-party deep neural network inference. USENIX Security 2022.
- [3] K. Yang, C. Weng, X. Lan, J. Zhang, X. Wang. Ferret: Fast extension for correlated OT with small communication. ACM CCS 2020.